

DSA5106 Deep Learning: Foundations and Techniques

AY2025/26 Sem2 By Zhao Peiduo

Lecture1

A Concrete Definition of Learning A computer program is said to **learn from experience** E with respect to a class of tasks T and performance measure P if its performance on tasks in T , as measured by P , improves with experience E .
Linear Regression as a Canonical Example Task (T):

- Inputs: $x \in \mathbb{R}^d$
- Outputs: $y \in \mathbb{R}$
- Underlying relationship: $y = f^*(x)$
- Goal: predict outputs given inputs

Experience (E):

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad y_i = f^*(x_i) \text{ or } y_i = f^*(x_i) + \varepsilon_i$$

Hypothesis Space Before defining performance, we specify a hypothesis space:

$$\mathcal{H} = \{\text{collection of candidate functions } f\}$$

Performance (P):

$$\text{Distance}(f, f^*)$$

Learning Objective Learning consists of selecting a hypothesis closest to the true function:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \text{Distance}(f, f^*)$$

Loss Functions and Empirical Risk Distance is defined using a loss function L over the dataset:

$$R_{\text{emp}}(f, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N L(f(x_i), y_i)$$

Example: Squared Loss

$$L(y', y) = \frac{1}{2} (y' - y)^2$$

Empirical Risk Minimization (ERM) Improving performance with experience is formulated as:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} R_{\text{emp}}(f, \mathcal{D})$$

Least Squares Formulation Define the design matrix and output vector:

$$X = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

ERM becomes:

$$\min_w \frac{1}{2N} \|Xw - y\|^2$$

Ordinary Least Squares Solution Setting the gradient to zero yields:

$$X^T(Xw - y) = 0$$

$$\hat{w} = (X^T X)^{-1} X^T y, \quad \hat{f}(x) = \hat{w}^T x$$

Extension to Affine Functions Linear models are often insufficient. A simple extension includes a bias term:

$$\mathcal{H} = \{f(x) = w^T x + b \mid w \in \mathbb{R}^d, b \in \mathbb{R}\}$$

Extension to Linear Basis Models To model nonlinear relationships, inputs are mapped into features:

$$\mathcal{H} = \left\{ f(x) = w^T \phi(x) = \sum_{i=1}^m w_i \phi_i(x) \right\}$$

The functions $\phi_1, \dots, \phi_m$ are called **basis functions** or **feature maps**, chosen *a priori*.

Examples of Basis Functions (1D)

- Polynomial basis: $\phi_j(x) = x^j$
- Gaussian basis: $\phi_j(x) = \exp\left(-\frac{(x-m_j)^2}{2s^2}\right)$
- Sigmoid basis: $\phi_j(x) = \sigma\left(\frac{x-m_j}{s}\right)$, $\sigma(b) = \frac{1}{1+e^{-b}}$

Universality of Linear Basis Models

The capacity of linear basis models depends on the **choice of basis functions** and their number. With appropriate basis functions and $m \rightarrow \infty$, linear basis models are **universal**: they can approximate arbitrarily well any target function (under mild conditions). For instance: Fourier series and Weierstrass approximation theorem

Least Squares for Linear Basis Models

The least squares solution is obtained by modifying the design matrix. Original design matrix:

$$X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix} \Rightarrow \Phi = \begin{pmatrix} \phi(x_1) \\ \phi(x_2) \\ \vdots \\ \phi(x_N) \end{pmatrix}$$

The empirical risk becomes:

$$R_{\text{emp}}(w) = \frac{1}{2N} \|\Phi w - y\|^2$$

Solution:

$$\hat{w} = (\Phi^T \Phi)^{-1} \Phi^T y$$

Model Capacity and Overfitting Using many basis functions greatly increases model flexibility:

$$\mathcal{H} = \left\{ f(x) = \sum_{j=0}^{99} w_j x^j \mid w \in \mathbb{R}^{100} \right\}$$

While such models can fit training data extremely well, they often exhibit **overfitting**: they capture noise rather than the true underlying relationship, leading to poor generalization.

Learning $\neq$ Optimization The goal of learning is not merely minimizing training error, but performing well on **unseen data**.

Empirical Risk Minimization (What we can solve):

$$\min_{f \in \mathcal{H}} R_{\text{emp}}(f) = \frac{1}{N} \sum_{i=1}^N L(f(x_i), f^*(x_i)), \quad x_i \sim \mu$$

Population Risk Minimization (What we want):

$$\min_{f \in \mathcal{H}} R_{\text{pop}}(f) = \mathbb{E}_{x \sim \mu} [L(f(x), f^*(x))]$$

The difference between these objectives is known as the **generalization gap**.

Estimating Population Risk

The true data distribution μ is unknown, but we observe samples through the dataset $\mathcal{D}$.

We therefore use a train-test split:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}$$

- $\mathcal{D}_{\text{train}}$: used for empirical risk minimization
- $\mathcal{D}_{\text{test}}$: used for final performance evaluation
- The test set must never influence training

Generalization

Generalization is a central challenge in machine learning. Key strategies to improve generalization include:

- Choice of model architecture
- Regularization
- Data augmentation
- Optimization methods

The Argmax View

Regression:

$$y = w^T \phi(x)$$

Classification (K classes):

$$y_i = w_i^T \phi(x), \quad i = 1, \dots, K$$

Prediction is obtained by:

$$\hat{y} = \arg \max_{i=1, \dots, K} y_i$$

This formulation is related to, but not equivalent to, support vector machines.

One-Hot Encoding and Softmax

Class labels are represented as one-hot vectors:

$$y_i = (0, \dots, 1, \dots, 0) \in \mathbb{R}^K$$

The model outputs class probabilities:

$$f(x_i) = (a_1, \dots, a_K) \in \mathbb{R}^K$$

Softmax Function

$$s(z)_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

Hypothesis space:

$$\mathcal{H} = \left\{ f(x) = s(W\phi(x)) \mid W \in \mathbb{R}^{K \times m} \right\}$$

The output can be interpreted as a probability distribution over K classes.

Loss Functions for Classification

Although classification is naturally defined using zero-one loss, it is not suitable for optimization.

Instead, we use surrogate losses:

$$L : \mathbb{R}^K \times \mathbb{R}^K \rightarrow \mathbb{R}_+$$

Desired properties:

- $L(y, y') = 0$ if $y = y'$
- $L(y, y')$ is small if $y \approx y'$
- L is differentiable (almost everywhere)

Examples of Loss Functions for Classification

Even in classification, we can apply the squared loss:

$$L(y, y') = \frac{1}{2} \|y - y'\|^2$$

While this loss is simple and differentiable, it is often poorly aligned with the probabilistic interpretation of classification outputs.

In particular:

- It does not strongly penalize confident but incorrect predictions
- It treats probability vectors as Euclidean points rather than distributions
- It may lead to slower or unstable optimization

Cross-Entropy Loss

A more appropriate loss for probabilistic classification is the cross-entropy loss:

$$L(y, y') = - \sum_{k=1}^K y'_k \log y_k$$

When y' is a one-hot encoded label, this loss penalizes low predicted probability assigned to the correct class.

General Perspective

Squared loss and cross-entropy are not the only possible choices.

Any differentiable measure of discrepancy between two probability distributions can be used as a surrogate loss, provided it supports efficient optimization.

Lecture 2

From Linear Basis Models to Neural Networks

Recall that linear basis models can be chosen to be **universal** with appropriate basis functions. Examples of 1D Bases include:

- Polynomial basis: $\phi(x) = (1, x, x^2, x^3, \dots)$
- Fourier basis: $\phi(x) = (1, \cos x, \sin x, \cos 2x, \sin 2x, \dots)$
- Wavelet basis (e.g. Haar wavelets)

Despite universality, empirical performance depends on the target function f^* , the basis choice ϕ , and the available computational budget m . **No fixed basis works well for all targets.**

Adaptive Basis Models

Standard linear basis models take the form:

$$f(x) = w^T \phi(x)$$

In adaptive models, the basis itself is learned:

$$f(x) = w^T \phi(x; \theta)$$

The parameters θ are learned from data to construct a basis well suited to the task.

The functions $\phi_i(\cdot; \theta)$ are also called **feature maps**, as they extract informative representations from data.

Neural Networks as Learned Feature Maps

A neural network can be viewed as a composition. The vector $h \in \mathbb{R}^m$ is the hidden representation, where m is the width of the hidden layer.

$$f = f^{(2)} \circ f^{(1)}$$

$$h = f^{(1)}(x; W, c), \quad y = f^{(2)}(h; w, b)$$

Need for Nonlinearity

If both layers are linear, the model collapses to a single linear function. Nonlinearity is introduced via an **activation function**:

$$h = g(Wx + c), \quad y = w^T h + b$$

The activation $g : \mathbb{R} \rightarrow \mathbb{R}$ is applied element-wise.

Common Activation Functions

- ReLU: $g(z) = \max(0, z)$
- Sigmoid: $g(z) = \frac{1}{1+e^{-z}}$
- Tanh: $g(z) = \tanh(z)$
- Leaky ReLU: $g(z) = \begin{cases} z, & z \geq 0 \\ \delta z, & z < 0 \end{cases}$

Shallow Neural Networks

A general one-hidden-layer network for regression is:

$$f(x; W, c, w, b) = w^T g(Wx + c) + b$$

where W, w are weights, c, b are biases, and g is the activation function.

Universal Approximation Theorem

A foundational result states that any continuous function on a compact domain can be approximated arbitrarily well by a neural network with a single hidden layer, provided the number of hidden units m is large enough.

Why Neural Networks over Linear Basis Models

A key limitation of fixed linear basis models is the **curse of dimensionality**: the number of required basis functions grows rapidly with input dimension, making learning inefficient. Neural networks mitigate this by learning compact, adaptive representations from data.

Neural Networks for Classification

Neural networks can be adapted for classification in the same way as linear models, by modifying the output layer.

Starting from a regression model:

$$f(x) = w^T g(Wx + c) + b$$

For multi-class classification, we use:

$$f(x) = \sigma(V g(Wx + c) + b), \quad V \in \mathbb{R}^{K \times m}$$

where σ is a normalization function producing class probabilities.

Choice of Output Units

- **Multiclass classification**: softmax output

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- **Multilabel classification**: sigmoid output applied independently

$$\text{sigmoid}(z)_i = \frac{1}{1 + e^{-z_i}}$$

Loss Functions

Learning is performed via **empirical risk minimization**:

$$R_{\text{emp}}(\theta) = \frac{1}{N} \sum_{i=1}^N L(f^*(x_i), f(x_i; \theta))$$

Regression

For regression problems, we typically use the squared loss:

$$L(y, y') = \frac{1}{2} (y - y')^2$$

Classification Losses

Multiclass classification: Cross-entropy loss

$$L(y, y') = - \sum_j y'_j \log y_j$$

Multilabel classification: Binary cross-entropy loss

$$L(y, y') = - \sum_j (y'_j \log y_j + (1 - y'_j) \log(1 - y_j))$$

Why Cross-Entropy over Square Loss

Cross-entropy is preferred for classification because:

- It has a clear probabilistic interpretation (maximum likelihood)
- It provides better numerical stability
- It works naturally with softmax outputs

Empirical Risk Minimization

Training a neural network amounts to solving:

$$\min_{\theta} R(\theta)$$

Unlike linear regression, closed-form solutions are generally unavailable.

Iterative Optimization

We therefore resort to iterative methods:

$$\theta_{k+1} = \theta_k + F(\theta_k), \quad R(\theta_{k+1}) \leq R(\theta_k)$$

Gradient Descent

A principled choice for the update direction is the negative gradient:

$$\theta_{k+1} = \theta_k - \epsilon \nabla R(\theta_k)$$

where $\epsilon > 0$ is a step size chosen to ensure a decrease in risk

Gradient Descent via Taylor Expansion

Consider a first-order Taylor expansion of the risk:

$$R(\theta + \epsilon \phi) = R(\theta) + \epsilon \phi^T \nabla R(\theta) + O(\epsilon^2)$$

To decrease the risk, we want the linear term $\phi^T \nabla R(\theta)$ to be as negative as possible. This is achieved by choosing:

$$\phi \propto -\nabla R(\theta)$$

Hence, parameters should be updated in the negative gradient direction:

$$\theta \leftarrow \theta - \epsilon \nabla R(\theta)$$

The Gradient Descent Algorithm

The direction $-\nabla R$ is called the **steepest descent direction**. This leads to the iterative algorithm:

$$\theta_{k+1} = \theta_k - \epsilon \nabla R(\theta_k)$$

This procedure is known as **Gradient Descent (GD)**. The scalar $\epsilon > 0$ is called the **step size** or **learning rate**.
Convergence Analysis (Linear Regression)
Consider gradient descent applied to linear regression. Let w_∞ denote the minimizer. The iteration can be rewritten as:

$$w_{k+1} - w_\infty = A(w_k - w_\infty), \quad A = I - \epsilon X^T X$$

Defining the error $e_k = w_k - w_\infty$, we obtain:

$$e_k = A^k e_0$$

Let $\{\lambda_i(A)\}$ be the eigenvalues of A . If

$$|\lambda_i(A)| < 1 \quad \forall i,$$

then $e_k \rightarrow 0$ and gradient descent converges. This condition is satisfied if:

$$\epsilon < \frac{2}{\max_i \lambda_i(X^T X)}$$

Rate of Convergence

Choosing

$$\epsilon = \frac{1}{\max_i \lambda_i(X^T X)},$$

yields the bound:

$$\|e_k\| \leq \left(1 - \frac{1}{\kappa(X^T X)}\right)^k \|e_0\|$$

where

$$\kappa(X^T X) = \frac{\max_i \lambda_i(X^T X)}{\min_i \lambda_i(X^T X)} \geq 1$$

is the **condition number**. Larger condition numbers imply slower convergence.

Gradient Descent for Neural Networks

For neural networks, empirical risk minimization becomes:

$$\min_{W, c, w, b} \sum_{i=1}^N L(f^*(x_i), f(x_i; W, c, w, b))$$

Closed-form solutions are unavailable, so the problem is solved iteratively using gradient descent:

$$W_{k+1} = W_k - \nabla_W R, \quad c_{k+1} = c_k - \nabla_c R, \quad \dots$$

This forms the basis of training neural networks via first-order optimization.

Lecture 3

From Shallow to Deep Networks

Shallow (One-Hidden-Layer) Neural Network

A shallow neural network with one hidden layer is defined by

$$h = g(Wx + b), \quad \hat{y} = w^\top h + c.$$

Deep Neural Networks

Deep networks are obtained by repeatedly composing hidden-layer transformations:

$$\begin{aligned} h^{(1)} &= g^{(1)}(W^{(1)}x + b^{(1)}), \\ h^{(2)} &= g^{(2)}(W^{(2)}h^{(1)} + b^{(2)}), \dots \\ h^{(\ell)} &= g^{(\ell)}(W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}), \\ \hat{y} &= w^\top h^{(\ell)} + c. \end{aligned}$$

Why Deep Neural Networks?

Although not fully understood theoretically, deep networks perform well due to:

- Efficient approximation of complex functions (e.g. highly oscillatory ones),
- Improved generalization with proper initialization, training, and regularization,
- Sequential feature extraction providing a strong inductive bias.

Issues with Deep Neural Networks

- Optimization is challenging due to non-convexity,
- Susceptibility to overfitting,
- High computational cost.

Structure of the Empirical Risk

The empirical risk is defined as

$$R(\theta) = \frac{1}{N} \sum_{i=1}^N R_i(\theta) = \frac{1}{N} \sum_{i=1}^N L\left(y^{(i)}, f(x^{(i)}; \theta)\right).$$

Example: Mean Squared Error (Regression)

$$R(\theta) = \frac{1}{N} \sum_{i=1}^N \left(y^{(i)} - w^\top \phi(x^{(i)}; \theta)\right)^2.$$

Computational Complexity of Gradient Descent

Gradient descent minimizes

$$R(\theta) = \frac{1}{N} \sum_{i=1}^N R_i(\theta)$$

via the update

$$\theta_{k+1} = \theta_k - \epsilon \frac{1}{N} \sum_{i=1}^N \nabla R_i(\theta_k).$$

- Per-iteration cost: $\mathcal{O}(N)$,
- Typical dataset sizes: $N \sim 10^4$ – 10^6 ,
- Goal: $\mathcal{O}(1)$ cost per iteration.

Stochastic Gradient Algorithm (Robbins & Monro, 1951)

Replace the full gradient with an unbiased estimate.

Gradient Descent (GD)

$$\theta_{k+1} = \theta_k - \epsilon \frac{1}{N} \sum_{i=1}^N \nabla R_i(\theta_k).$$

Stochastic Gradient Descent (SGD)

Sample

$$\gamma_k \sim \text{Uniform}\{1, \dots, N\},$$

and update

$$\theta_{k+1} = \theta_k - \epsilon \nabla R_{\gamma_k}(\theta_k).$$

Unbiasedness

$$\mathbb{E}[\theta_{k+1} \mid \theta_k] = \theta_k - \epsilon \frac{1}{N} \sum_{i=1}^N \nabla R_i(\theta_k).$$

Mini-Batch SGD

Use a subset $B_k \subset \{1, \dots, N\}$ with $|B_k| = M$:

$$\theta_{k+1} = \theta_k - \epsilon \frac{1}{M} \sum_{j \in B_k} \nabla R_j(\theta_k).$$

Computational Complexity

$$\begin{aligned} \text{GD} &: \mathcal{O}(N), \\ \text{SGD} &: \mathcal{O}(1), \\ \text{Mini-batch SGD} &: \mathcal{O}(M). \end{aligned}$$

Disadvantages of SGD

- GD converges to a stationary point for sufficiently small ϵ ,
- If $R(\theta)$ is convex, GD converges to the global minimum,
- Question: what guarantees (if any) hold for SGD?

Example: SGD Dynamics

Recall the gradient descent (GD) dynamics for the quadratic risk

$$R(\theta) = \frac{1}{2} \|\theta\|^2,$$

which yields

$$\theta_k = (1 - \epsilon)^k \theta_0 \rightarrow 0 \quad \text{if } 0 < \epsilon < 2.$$

1D Stochastic Variant

Consider a one-dimensional empirical risk:

$$R(\theta) = \frac{1}{N} \sum_{i=1}^N R_i(\theta), \quad R_i(\theta) = \frac{1}{2} (\theta - \theta^{(i)})^2,$$

with assumptions

$$\frac{1}{N} \sum_i \theta^{(i)} = 0, \quad \frac{1}{N} \sum_i (\theta^{(i)})^2 = 1.$$

The gradient for sample i is

$$\nabla R_i(\theta) = \theta - \theta^{(i)}.$$

SGD Iteration

With $\gamma_k \sim \text{Uniform}\{1, \dots, N\}$,

$$\theta_{k+1} = \theta_k - \epsilon \nabla R_{\gamma_k}(\theta_k) = (1 - \epsilon)\theta_k + \epsilon \theta^{(\gamma_k)}.$$

Solving recursively, where the first term is deterministic and the second is random:

$$\theta_k = (1 - \epsilon)^k \theta_0 + \epsilon \sum_{j=1}^k (1 - \epsilon)^{j-1} \theta^{(\gamma_{k-j})},$$

Mean Behavior

Taking expectations,

$$\mathbb{E}[\theta_k] = (1 - \epsilon)^k \theta_0,$$

so on average SGD follows the same dynamics as GD.

Variance Behavior

Using the i.i.d. assumption,

$$\mathbb{E}[\theta^{(\gamma_{k-j})}] = 0, \quad \mathbb{E}[\theta^{(\gamma_{k-j})} \theta^{(\gamma_{k-\ell})}] = \delta_{j\ell},$$

we obtain

$$\mathbb{E}[\theta_k^2] = (1 - \epsilon)^{2k} \theta_0^2 + \frac{\epsilon}{2 - \epsilon} [1 - (1 - \epsilon)^{2k}].$$

As $k \rightarrow \infty$,

$$\mathbb{E}[\theta_k] \rightarrow 0, \quad \mathbb{E}[\theta_k^2] \rightarrow \frac{\epsilon}{2 - \epsilon},$$

showing persistent fluctuations for fixed ϵ .

SGD with Decaying Learning Rate

With a varying step size,

$$\theta_{k+1} = \theta_k - \epsilon_k \nabla R_{\gamma_k}(\theta_k),$$

SGD converges if the learning rate decays appropriately. A sufficient condition is

$$\sum_{k=0}^{\infty} \epsilon_k = \infty, \quad \sum_{k=0}^{\infty} \epsilon_k^2 < \infty.$$

Variants of SGD Common extensions include: **Momentum**, adaptive learning rates (Adagrad, Adadelata), combined methods (Adam, RMSProp), variance reduction (SVRG)

Momentum (Polyak, 1964) Momentum accelerates GD/SGD by remembering past descent directions. For

$$R(\theta) = \frac{1}{2} \theta_1^2 + \frac{\lambda}{2} \theta_2^2, \quad \lambda \gg 1,$$

the gradient is

$$\nabla R(\theta) = \begin{pmatrix} \theta_1 \\ \lambda \theta_2 \end{pmatrix}.$$

Momentum updates are

$$v_{k+1} = \alpha v_k - \epsilon \nabla R(\theta_k), \quad \theta_{k+1} = \theta_k + v_{k+1}.$$

where v_k is the momentum and $\alpha \in (0, 1)$ is the momentum parameter.

- $\alpha = 0$: recovers GD
- $\alpha > 0$: incorporates past directions to reduce zig-zag behavior

Physical Interpretation

Momentum is inspired by Newton's second law:

$$p = mv, \quad F = m \frac{dv}{dt}.$$

With friction proportional to velocity, which corresponds discretely to momentum SGD.

$$\frac{dv}{dt} = -\frac{\gamma}{m} v + \frac{1}{m} F_{\text{ext}}(\theta), \quad \frac{d\theta}{dt} = v,$$

SGD in Practice

In practice, SGD almost always uses mini-batches, where $B_k \subset \{1, \dots, N\}$ with $|B_k| = M$:

$$\theta_{k+1} = \theta_k - \epsilon \frac{1}{M} \sum_{j \in B_k} \nabla R_j(\theta_k),$$

The dataset is typically traversed in chunks of size M ; one full pass is called an **epoch**.

Computing Gradients

In Gradient Descent (GD) and Stochastic Gradient Descent (SGD), we need to compute gradients of the empirical risk:

$$\nabla R(\theta) \quad \text{or} \quad \nabla R_i(\theta).$$

A key question is how to compute these gradients efficiently for deep neural networks.

Computational Graph

A *computational graph* is a directed graph where:

- Nodes represent variables,
- Directed edges represent operations.

Computational graphs allow us to organize and reuse intermediate computations.

Chain Rule for Scalar Functions

Let $x \in \mathbb{R}$ and f, g be scalar functions. Define

$$y = g(x), \quad z = f(y) = f(g(x)).$$

Then the chain rule gives

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx} = f'(g(x)) g'(x).$$

Chain Rule for Vector-Valued Functions

Let

$$f: \mathbb{R}^m \rightarrow \mathbb{R}^n, \quad g: \mathbb{R}^n \rightarrow \mathbb{R}, \quad y = f(x), \quad z = g(y).$$

Then

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}.$$

In vector notation,

$$\nabla_x z = \left(\frac{\partial y}{\partial x} \right)^T \nabla_y z,$$

where $\frac{\partial y}{\partial x}$ is the Jacobian of f .

Why Do We Need an Algorithm?

Although the chain rule tells us how to compute gradients, applying it naively leads to repeated and inefficient computations.

We want an **efficient algorithm** that avoids unnecessary recomputation and scales to deep networks.

Example: Simplest Backpropagation

Consider a 1D linear neural network with one data point:

$$h^{(1)} = w^{(1)} x, \quad h^{(2)} = w^{(2)} h^{(1)}, \quad h^{(3)} = w^{(3)} h^{(2)}, \quad \hat{y} = w h^{(3)}.$$

All variables are scalars. The empirical risk is

$$R(\theta) = R(w^{(1)}, w^{(2)}, w^{(3)}, w) = L(\hat{y}(x; \theta), y).$$

Goal: compute $\frac{\partial R}{\partial w^{(j)}}$ for $j = 1, 2, 3$ and $\frac{\partial R}{\partial w}$ efficiently.

Step 1: Forward Propagation

Given input x , label y , and current weights:

$$h^{(1)} = w^{(1)} x, \quad h^{(2)} = w^{(2)} h^{(1)}, \quad h^{(3)} = w^{(3)} h^{(2)}, \quad \hat{y} = w h^{(3)}.$$

Store intermediate values $h^{(j)}$ for backpropagation.

Step 2: Backward Propagation (Hidden States)

Last layer:

$$\frac{dR}{d\hat{y}} = \frac{\partial}{\partial \hat{y}} L(y, \hat{y}) \rightarrow \hat{p}$$

Next layers:

1. $\frac{dR}{dh^{(3)}} = \frac{dR}{d\hat{y}} \frac{d\hat{y}}{dh^{(3)}} = \hat{p} w \rightarrow p^{(3)}$
2. $\frac{dR}{dh^{(2)}} = \frac{dR}{dh^{(3)}} \frac{dh^{(3)}}{dh^{(2)}} = p^{(3)} w^{(3)} \rightarrow p^{(2)}$
3. $\frac{dR}{dh^{(1)}} = \frac{dR}{dh^{(2)}} \frac{dh^{(2)}}{dh^{(1)}} = p^{(2)} w^{(2)} \rightarrow p^{(1)}$

Step 3: Computing Derivatives wrt Parameters

By Chain Rule:

$$\frac{dR}{dw} = \frac{dR}{d\hat{y}} \frac{d\hat{y}}{dw} = \hat{p} h^{(3)}$$

And for $j = 3, 2, 1$:

$$\frac{dR}{dw^{(j)}} = \frac{dR}{dh^{(j)}} \frac{dh^{(j)}}{dw^{(j)}} = p^{(j)} h^{(j-1)} \quad (\text{where } h^{(0)} := x)$$

Notable Properties

- Two passes: one forward and one backward.
- Total cost: $\mathcal{O}(\#\text{nodes})$ (naïve rule is quadratic).
- Only forward states need to be stored.

Generalizations

- Non-linear and Multi-dimensional cases.
- Tensor inputs/outputs & general DAGs.
- Multi-sample: $\sum \text{backprop}(R_i)$.

Lecture 4

Permutations

A **permutation** of n objects is a one-to-one transformation on these objects. Equivalently, a permutation is a **bijection** on the index set $\{1, 2, \dots, n\}$.

Permutation Invariance of Hypothesis Spaces

Observe that the fully connected neural network (FCNN) hypothesis space $\mathcal{H}$ satisfies the following invariance property: If $f \in \mathcal{H}$ and p is any permutation of the indices, define

$$f_p(x_1, \dots, x_d) \equiv f(x_{p(1)}, \dots, x_{p(d)}).$$

Then, $f_p \in \mathcal{H}$ as well.

Limitation of FCNNs

Permutation invariance implies that FCNNs **discard spatial and temporal structure** in the data. This makes them poorly suited for signals, images, and time series where locality matters.

Averaged Measurement

To reduce noise, we average measurements using a weighting function:

$$s(t) = \int_{-\infty}^{+\infty} x(a) w(t-a) da.$$

For example, an exponential kernel:

$$w(t-a) = \begin{cases} e^{-(t-a)}, & t \geq a, \\ 0, & t < a. \end{cases}$$

Convolution of Functions

The weighted average above is a special case of a **convolution**. Given two real-valued functions $x(t)$ and $w(t)$, their convolution is

$$s(t) = (x * w)(t) = \int_{-\infty}^{+\infty} x(a) w(t-a) da.$$

Properties of Convolution

Convolution is **commutative**:

$$x * w = w * x.$$

Conventionally:

- $x(t)$ is called the **signal** or input,
- $w(t)$ is called the **kernel** or filter,
- $s(t) = x * w$ is called the **feature map**.

Discrete Convolutions

In practice, measurements are taken at discrete times: $t = \dots, -1, 0, 1, 2, \dots$. The discrete convolution is defined as

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{+\infty} x(a) w(t-a).$$

Finite Convolutions and Boundary Conditions

Real signals are finite, so convolutions must be truncated. Common boundary-handling strategies include:

- Circular convolutions,
- Valid convolutions,
- Zero-padded convolutions with **same** padding.

2D Discrete Convolutions

For image and vision tasks, signals are represented as 2D matrices. Given an input image I and kernel K , the 2D convolution is

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i-m, j-n).$$

Convolution remains commutative:

$$I * K = K * I.$$

Cross-Correlation vs Convolution

Convolution:

$$S(i, j) = \sum_m \sum_n I(i-m, j-n) K(m, n).$$

Cross-correlation:

$$S(i, j) = \sum_m \sum_n I(i+m, j+n) K(m, n).$$

While the numerical values may coincide, cross-correlation is not commutative. Deep learning libraries typically implement cross-correlation but refer to it as convolution.

Tensor Convolution

When the input has multiple channels, each filter has the same number of channels. Each channel is convolved separately and the results are summed:

$$\# \text{ output channels} = \# \text{ filters}.$$

Convolution as a Linear Operation

A zero-padded 1D convolution can be written as

$$w * x = A(w)x,$$

where $A(w)$ is a structured (Toeplitz) matrix. Thus, convolutions are linear operations.

Convolutional Neural Networks

CNNs replace fully connected layers with convolutions:

$$h^{(i+1)} = g\left(w^{(i+1)} * h^{(i)} + b^{(i+1)}\right).$$

Motivation 1: Sparse Interactions

Due to spatial locality, each neuron interacts with only a small neighborhood, unlike fully connected layers where every node interacts with every other node.

Implicit Connectivity

Although each convolutional layer is local, stacking multiple layers results in long-range dependencies being captured implicitly.

Computational Efficiency

Let image size be $m \times n$ and kernel size be $k \times l$. The computational cost is:

- CNN: $\mathcal{O}(klmn)$,
- FCNN: $\mathcal{O}(m^2n^2)$.

Since $k, l \ll m, n$, CNNs are significantly more efficient.

Motivation 2: Parameter Sharing

Parameter sharing refers to using the same trainable parameters at multiple spatial locations in a model. An alternative name is **tied weights**. In 1D convolution, this leads to a structured linear operator

$$w * x = A(w)x,$$

$$A(w) = \begin{pmatrix} w_2 & w_3 & 0 & 0 & 0 \\ w_1 & w_2 & w_3 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \\ 0 & 0 & 0 & w_1 & w_2 \end{pmatrix}$$

has only **3 degrees of freedom**, compared to a fully connected matrix.

Memory Efficiency

For an image of size $m \times n$ and kernel size $k \times l$:

- With parameter sharing: $\mathcal{O}(k \times l)$ parameters,
- Without sharing: $\mathcal{O}(m \times n \times k \times l)$ parameters.

Motivation 3: Equivariance and Invariance

Let f, g be functions.

- f is **equivariant** w.r.t. g if

$$f(g(x)) = g(f(x)).$$

- f is **invariant** w.r.t. g if

$$f(g(x)) = f(x).$$

Convolutions and Translations

Let T_τ be a translation operator:

$$T_\tau(x)(t) = x(t - \tau).$$

Then for any kernel w ,

$$w * T_\tau(x) = T_\tau(w * x),$$

showing that **convolutions are equivariant to translations**.

Compositions of Equivariant Maps

If f_1, f_2 are equivariant w.r.t. g , then so is $f_2 \circ f_1$:

$$f_2 \circ f_1(g(x)) = g(f_2 \circ f_1(x)).$$

Hence each convolutional layer

$$h^{(i+1)} = \sigma(w * h^{(i)} + b)$$

is translation equivariant.

Infinitely Strong Priors

$$\mathcal{H}_{\text{inv}} = \{ f : f(T(x)) = f(x) \} \quad f^* \in \mathcal{H}_{\text{inv}}$$

Building an Invariant Mapping

Let $f_1, \dots, f_\ell$ be equivariant w.r.t. g , and let F be invariant w.r.t. g . Then

$$F \circ f_\ell \circ \dots \circ f_1$$

is invariant w.r.t. g .

A deep CNN of the following form is therefore translation invariant:

$$h^{(i+1)} = \sigma(W * h^{(i)} + b), \quad h^{(0)} = x,$$

$$\hat{y} = \mathbf{1}^\top h^{(\ell)} + c$$

Importance of Translation Invariance

For image classification, the target function f^* must satisfy

$$f^*(T(I)) = f^*(I),$$

since translated images of the same object share the same label. CNNs encode this inductive bias directly.

Pooling Layers

Pooling builds **approximate invariance** to small translations and deformations. Common types include **max** and **mean** pooling.

Basic Structure of CNNs

A convolutional block typically consists of: Convolution (affine transform), nonlinearity (e.g. ReLU), pooling.

Variants of Convolutions

Common extensions include strided convolutions, up/down-sampling operations, unshared convolutions, channel-wise / separable convolutions.

Lecture 5

What is a time series?

Like images, a time series is another type of data with some local structure.

Instead of spatial structure in images, we consider temporal structures.

Representations of a Time Series

We may represent a time series as either:

- A discrete sequence

$$x^{(1)}, x^{(2)}, x^{(3)}, \dots$$

- A continuous sequence

$$\{x^{(t)} : t \geq 0\}$$

Each slice of the time series, $x^{(t)}$, is a vector, or something that can be represented as a vector.

Task I: Sequence Prediction

Given a historical time series data:

$$x^{(1)}, x^{(2)}, \dots, x^{(t)}$$

Task is to predict:

- The next data point $x^{(t+1)}$
- A sequence of next data points

$$x^{(t+1)}, x^{(t+2)}, \dots, x^{(t+\tau)}$$

Example applications:

- Stock price prediction, weather forecasting

Task II: Sequence Classification/Regression

Given a time series data:

$$x^{(1)}, x^{(2)}, \dots, x^{(t)}$$

Task is to predict a label y of this sequence, which can be either discrete (classification) or continuous (regression).

Example Applications:

- Credit card transaction fraud detection
- Heart arrhythmia detection

Task III: Sequence-to-sequence Modelling

Given a time series data:

$$x^{(1)}, x^{(2)}, \dots, x^{(t)}$$

Task is to predict a corresponding time series:

$$y^{(1)}, y^{(2)}, \dots, y^{(t)}$$

Example Applications:

- Machine translation
- Continuous health monitoring using wearable devices

Task IV: Sequence Generation

Given a seed sequence:

$$x^{(1)}, x^{(2)}, \dots, x^{(\tau)}$$

Task is to generate a longer sequence starting from this, i.e.

$$x^{(1)}, x^{(2)}, \dots, x^{(\tau)}, x^{(\tau+1)}, x^{(\tau+2)}, \dots$$

according to some distribution.

Example applications:

- Writing poems
- Composing music

The Supervised Learning Problem

Recall that in supervised learning, we define the inputs, outputs and the target function that maps the former to the latter.

For time series modelling, we can define these similarly:

- Inputs: $x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(\tau)}$
- Outputs: $y^{(1)}, y^{(2)}, y^{(3)}, \dots, y^{(\tau)}$
- Target: $\{F_t^*\}$ with

$$y^{(t)} = F_t^*(x^{(1)}, x^{(2)}, \dots, x^{(\tau)})$$

Goal: Learn some $\{\hat{F}_t\}$ to approximate $\{F_t^*\}$.

Examples

Task I: Sequence Prediction

$$F_t^*(x^{(1)}, x^{(2)}, \dots, x^{(\tau)}) = F_t^*(x^{(1)}, x^{(2)}, \dots, x^{(t)}) = x^{(t+1)}$$

Task II: Sequence Classification/Regression

$$F_\tau^*(x^{(1)}, x^{(2)}, \dots, x^{(\tau)}) = y^{(\tau)} = y$$

Task III: Sequence-to-sequence

$$F_t^*(x^{(1)}, x^{(2)}, \dots, x^{(\tau)}) = y^{(t)}, \quad t = 1, 2, \dots$$

or

$$F_t^*(x^{(1)}, x^{(2)}, \dots, x^{(t)}) = y^{(t)}, \quad t = 1, 2, \dots$$

Dynamical System

A natural way to share parameters in time is to define a dynamical system depending on some parameter:

$$s^{(t+1)} = f(s^{(t)}; \theta)$$

- The vector $s^{(t)}$ is the state of the dynamical system at time t
- θ is a vector of parameters

This is different from “feed-forward” dynamics:

$$h^{(t+1)} = f(h^{(t)}; \theta^{(t)})$$

i.e. the parameter θ is shared in time.

Computational Graph Representation

We can represent the dynamics

$$s^{(t+1)} = f(s^{(t)}; \theta)$$

using the usual computational graph approach.

Unrolling the Dynamics

Of course, we can also unroll the dynamics:

$$s^{(t)} = f(s^{(t-1)}; \theta) = f(f(s^{(t-2)}; \theta); \theta)$$

$$= \dots = f(f(\dots f(s^{(0)}; \theta) \dots; \theta); \theta).$$

Dynamical Systems with Inputs

Now, we consider a dynamical system with inputs or external forcing:

$$s^{(t)} = f(s^{(t-1)}, x^{(t)}, \theta).$$

Recurrent Neural Networks

The basic architecture of recurrent neural networks uses a forced, hidden dynamical system as a basic hypothesis space:

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}, \theta)$$

$$\hat{y}^{(t)} = g(h^{(t)}, \phi).$$

This implicitly parameterizes

$$\hat{F}_t(x^{(1)}, x^{(2)}, \dots, x^{(t)}; \theta, \phi) = \hat{y}^{(t)}.$$

Simple RNN

To be explicit, we consider the following simple RNN structure:

$$h^{(t)} = \sigma_r(W h^{(t-1)} + U x^{(t)} + b)$$

$$\hat{y}^{(t)} = o^{(t)} = \sigma_o(V h^{(t)} + c).$$

- The trainable parameters are

$$(\theta, \phi) = (W, U, b, V, c).$$

- The recurrent activation σ_r is usually taken to be **tanh**.
- Initialize $h^{(0)} = 0$ by default.
- The output activation function σ_o depends on application.

Tanh Gradients

- Gradient peaks at 1 when $a = 0$.
- Can saturate to ± 1 .

Sigmoid Gradients

- Smaller peak gradient of 0.25.
- Non-zero center, outputs saturate to positive numbers, causing systematic bias accumulation.

ReLU Gradients

- Unbounded gradient.
- Dead neurons are dead forever because same weights are shared across time.

Stable Identity Dynamics of tanh

Simple scalar example, $\sigma = \tanh$:

$$h^{(t)} = \sigma(w h^{(t-1)})$$

$$\frac{\partial h^{(t)}}{\partial h^{(t-1)}} = \sigma'(w h^{(t-1)}) \cdot w.$$

When $a \approx 0$, $\sigma(a) \approx a$ and $\sigma'(a) \approx 1$.

If $h^{(t)}$ is small and $w \approx 1$: $h^{(t)} \approx h^{(t-1)}$, Gradients approx. 1

Loss Functions

For single-prediction tasks (Tasks I and II), the loss function may be defined only at the end of the sequence, e.g.

$$L(y, \hat{y}^{(\tau)}) = \frac{1}{2} \|y - \hat{y}^{(\tau)}\|^2.$$

For sequence-prediction tasks (Task III), we may take the sum:

$$\sum_{t=1}^{\tau} L(y^{(t)}, \hat{y}^{(t)}) = \frac{1}{2} \sum_{t=1}^{\tau} \|y^{(t)} - \hat{y}^{(t)}\|^2.$$

Why a hidden dynamical system?

Consider instead the naive approach of just using FCNN to map each input to each output:

y^(t) = FCNN(x^(t)).

What is wrong?

- The prediction at time t only depends on x^(t).
- This cannot model systems with memory, no matter how complex FCNN is.

Unrolling Computational Graph

Once we unroll the computational graph, then RNN is just a parameter-tied feed-forward NN, thus we can apply back-propagation algorithm.

General BPTT

General Back-propagation through time (BPTT) basically does the same:

- Unroll computational graph
- Compute “deltas” represented by

p^(t) := ∇_h(t) L

- Compute derivatives with respect to weights by **summing** over contributions at each time step

Only the last step differs from the feed-forward case, due to parameter sharing in time.

Gradient Explosion and Vanishing

A big problem that plagues RNNs is the difficulty to train them.

One is often faced with a dilemma for large τ:

- Gradient explosion: for some parameter values, the gradients computed using the RNN diverges
- Gradient vanishing: for other parameter values, the gradients computed using the RNN gives vanishing weight to faraway inputs

Gating and Long-Term Dependencies

One of the most effective ways to overcome the inability for RNNs to learn long-term dependencies is their gated extensions.

The key ideas:

- Design RNN variants where gradients neither vanish nor explode
- Achieve this by the construction of “trainable gates”, which controls the flow of information
 - Gates control the accumulation of information
 - When accumulated information is no-longer required, we just forget them

RNN Cells

RNNs operate by feeding the input and previous hidden states to a function to give the next hidden state, e.g.

h^(t) = tanh(W h^(t-1) + U x^(t) + b)

We can abstract this operation as a black-box cell A.

Long Short Term Memory (LSTM)

LSTM [Hochreiter and Schmidhuber, 1997] is one of the most commonly used gated RNNs.

The Cell State

The cell state C_t is another hidden variable that is designed to flow through time with minimal interruptions.

Gates

The LSTM interacts with the cell state using gates, there are just layers with sigmoid outputs

u ↦ σ(Wu + b),

σ(z) = 1 / (1 + e^-z) ∈ (0, 1)

Multiplying the cell state with the above makes a modification to it.

Summary: LSTM Model (In our Notation)

f^(t) = σ(W_f h^(t-1) + U_f x^(t) + b_f)

i^(t) = σ(W_i h^(t-1) + U_i x^(t) + b_i)

o^(t) = σ(W_o h^(t-1) + U_o x^(t) + b_o)

C^(t) = f_t · C_{t-1} + i_t · σ_C(W_c h^(t-1) + U_c x^(t) + b_c)

h^(t) = o^(t) · σ_h(C^(t))

Gates: f^(t), i^(t), o^(t)

Cell state: C^(t)

Hidden/RNN state: h^(t)

Variants of the LSTM Architecture

There are many variants:

- Peephole connections
- Coupled input/forget gates
- Gated Recurrent Unit (GRU)

Deep Recurrent Neural Networks

One may observe that the RNN, even in its simplest form, is already deep in the time direction.

However, each time-step relied on a single layer of FCNN-like structure.

Deep RNNs generalize this by using a deep NN for the recurrent step.

The Basic Architecture

Shallow/Simple RNN:

h^(t) = σ_r(W h^(t-1) + U x^(t) + b)

y^(t) = σ_o(V h^(t) + c)

Deep RNN (One variant):

h^(t) = σ_r(W h^(t-1) + U x^(t) + b)

z^(t) = σ_r(S z^(t-1) + T h^(t) + a)

y^(t) = σ_o(V z^(t) + c)

Other Ways to Obtain Deep RNN Models

- (a) Using stacked hidden/cell states
- (b) Using MLPs for connections

Other Important Architectures for Sequence Modelling

There are a number of other architectures for time-series modelling that are useful to know:

- Bi-directional RNNs
- Sequence-to-sequence autoencoders
- Attention mechanisms and the transformer network

Lecture 6

Regularization Regularization is a general technique: Any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error. In other words, we trade **some bias to lower variance**.

Parameter Norm Penalties Empirical risk minimization

θ̂ = arg min_θ R(θ; X, y)

A parameter norm penalty modifies this to

R̂(θ; X, y) = R(θ; X, y) + αΩ(θ)

where

- Ω is called a regularizer
- α ≥ 0 is the strength (coefficient) of regularization

L2 Regularization

The simplest parameter norm regularization is the L2 regularizer

Ω(θ) = 1/2 ||θ||^2 = 1/2 ∑_i θ_i^2

so that

R̂(θ; X, y) = R(θ; X, y) + 1/2 α ||θ||^2

and

θ̂ = arg min_θ R̂(θ; X, y)

Example: Linear Regression

L2-regularized linear model

R(w; X, y) = 1/2 ||Xw - y||^2 + 1/2 α ||w||^2

This is called **ridge regression** (Tikhonov regularization).

Solution:

ŵ = (X^T X + αI)^-1 X^T y

Compare with unregularized version:

H = X^T X, Ĥ = H + αI

Effect of the Regularization

Recall

1/m H_ij = 1/m (X^T X)_ij = 1/m ∑_k x_i^(k) x_j^(k)

which is the covariance matrix of the data.

The principal directions are eigenvectors {u_i} of H with eigenvalues {λ_i}.

Let

H u_i = λ_i u_i

Expand

X^T y = ∑_i β_i u_i

Unregularized:

ŵ = ∑_i β_i / λ_i u_i

Regularized:

$$\hat{w} = \sum_i \frac{\beta_i}{\lambda_i + \alpha} u_i$$

Effect of L^2 Regularization

- If $\lambda_i \gg \alpha$, then $\lambda_i + \alpha \approx \lambda_i$
- If $\lambda_i \ll \alpha$, then $\lambda_i + \alpha \approx \alpha$
- The regularization removes the influence of the data in directions of small variance.

Additional Remarks on L^2 Regularization

L^2 regularization is also called **weight decay**.

Gradient descent:

$$\theta_{k+1} = \theta_k - \epsilon \nabla \bar{R}(\theta_k) = \theta_k - \epsilon \nabla R(\theta_k) - \epsilon \alpha \theta_k$$

If $R = C$ (constant):

$$\theta_{k+1} = (1 - \epsilon \alpha) \theta_k$$

L^1 Regularization

Another type of parameter norm regularization is L^1 :

$$\Omega(\theta) = \|\theta\|_1 = \sum_i |\theta_i|$$

L^1 vs L^2 Regularization

Consider minimizing

$$R(\theta) = \frac{1}{2} \sum_i \lambda_i (\theta_i - \theta_i^*)^2$$

where $\theta^* \in \mathbb{R}^m$ is fixed.

Solution for L^2 regularized case

$$\nabla \bar{R}(\hat{\theta}) = 0 \Rightarrow \lambda_i (\hat{\theta}_i - \theta_i^*) + \alpha \hat{\theta}_i = 0 \Rightarrow \hat{\theta}_i = \frac{\lambda_i}{\lambda_i + \alpha} \theta_i^*$$

Solution for L^1 regularized case

$$\bar{R}(\theta) = \sum_i R_i(\theta_i)$$

with

$$R_i(\theta_i) = \frac{1}{2} \lambda_i (\theta_i - \theta_i^*)^2 + \alpha |\theta_i|$$

Thus we can minimize each component of θ independently:

$$\hat{\theta}_i = \begin{cases} \theta_i^* - \text{Sign}(\theta_i^*) \frac{\alpha}{\lambda_i}, & |\theta_i^*| > \frac{\alpha}{\lambda_i} \\ 0, & |\theta_i^*| \leq \frac{\alpha}{\lambda_i} \end{cases}$$

Unregularized

$$\hat{\theta}_i = \theta_i^*$$

L^2 -Regularized

$$\hat{\theta}_i = \frac{\lambda_i}{\lambda_i + \alpha} \theta_i^*$$

L^1 -Regularized

$$\hat{\theta}_i = \begin{cases} \theta_i^* - \text{Sign}(\theta_i^*) \frac{\alpha}{\lambda_i}, & |\theta_i^*| > \frac{\alpha}{\lambda_i} \\ 0, & |\theta_i^*| \leq \frac{\alpha}{\lambda_i} \end{cases}$$

Comparison between L^2 and L^1 regularization

Common to both:

- As α increases from 0 to ∞ , the solution $\hat{\theta}$ is pushed from θ^* toward 0
- Higher variance components of the data (large λ_i) are less affected than low variance components (small λ_i)

Differences:

- L^1 is “hard”: if λ_i is small enough the corresponding weight becomes exactly 0
- L^1 penalty therefore induces sparsity

Remarks on L^1 and L^2 Regularization

- Sparsity can be used for feature selection (principle behind LASSO)
- From a Bayesian MAP perspective, parameter norm regularization corresponds to priors on parameters

$$L^2 : \quad p(\theta) \propto \exp\left(-\frac{\alpha}{2} \|\theta\|^2\right)$$

$$L^1 : \quad p(\theta) \propto \exp(-\alpha \|\theta\|_1)$$

Elastic Net combines both L^1 and L^2 penalties.

Remarks on Regularizing Neural Networks

So far examples were linear models, but regularization behaves similarly for nonlinear neural networks.

Consider a neural network layer:

$$\sigma(Wx + b)$$

Usually we regularize weights W but not bias b .

Bias terms only shift activations and do not need to be small.

Different layers may use different regularization strengths.

Training vs Validation Loss

Often in practice:

- Training loss continues decreasing
- Validation loss stays constant or increases

This indicates overfitting.

Early Stopping Variants

Early stopping requires a validation set.

Two strategies:

- Retrain on the full dataset after identifying optimal number of epochs
- Continue training on full dataset using the validation loss threshold

Algorithm: Early Stopping (Retrain Variant)

Let $X^{(train)}, y^{(train)}$ be the training set.

Split into

$$(X^{(subtrain)}, X^{(valid)}), \quad (y^{(subtrain)}, y^{(valid)})$$

Run early stopping using $X^{(subtrain)}$ and $y^{(subtrain)}$ for training and validation data for monitoring.

This returns optimal number of steps i^* .

Reinitialize parameters θ .

Train on full training data for i^* steps.

Algorithm: Early Stopping (Continuation Variant)

Let $X^{(train)}, y^{(train)}$ be the training set.

Split into

$$(X^{(subtrain)}, X^{(valid)}), \quad (y^{(subtrain)}, y^{(valid)})$$

Run early stopping to update θ .

Set

$$\epsilon \leftarrow J(\theta, X^{(subtrain)}, y^{(subtrain)})$$

While

$$J(\theta, X^{(valid)}, y^{(valid)}) > \epsilon$$

train on full dataset for n steps.

Example: Early Stopping vs Explicit Regularization

Consider the one-dimensional loss

$$R(\theta) = \frac{1}{2} \lambda (\theta - \theta^*)^2$$

Gradient descent

$$\theta_{k+1} = \theta_k - \epsilon \lambda (\theta_k - \theta^*) = (1 - \epsilon \lambda) \theta_k + \epsilon \lambda \theta^*$$

Thus

$$\theta_k = (1 - \epsilon \lambda)^k \theta_0 + \left(1 - (1 - \epsilon \lambda)^k\right) \theta^*$$

If we stop at iteration τ :

$$\hat{\theta} = (1 - \epsilon \lambda)^\tau \theta_0 + \left(1 - (1 - \epsilon \lambda)^\tau\right) \theta^*$$

Hence $\hat{\theta}$ is a weighted average of the initial parameter θ_0 and the optimum θ^* .

Let us now look at a variant of L^2 regularized GD

$$\bar{R}(\theta) = R(\theta) + \frac{1}{2} \alpha (\theta - \theta_0)^2$$

That is, we apply the regularization centered at θ_0 .

Then the solution is

$$\hat{\theta} = \frac{\alpha}{\alpha + \lambda} \theta_0 + \left(1 - \frac{\alpha}{\alpha + \lambda}\right) \theta^*$$

Compare with early stopping

$$\hat{\theta} = (1 - \epsilon \lambda)^\tau \theta_0 + \left(1 - (1 - \epsilon \lambda)^\tau\right) \theta^*$$

In other words, early stopping at time τ is equivalent to L^2 regularization with strength

$$\alpha = \frac{\lambda(1 - \epsilon \lambda)^\tau}{1 - (1 - \epsilon \lambda)^\tau}$$

Remarks on Early Stopping

Advantages:

- Very easy to implement
- Effectively a regularization with a penalty depending on the distance from initial condition

$$\frac{1}{2} \alpha (\theta - \theta_0)^2$$

- Implicit regularization, no need to choose α .

Different ways to add Noise

The basic rationale behind adding noise to regularize models:

Our model should be robust to noise

Different ways to add noise:

- Inputs
- Outputs / labels
- Weights

Adding Noise to Inputs

This is heuristically equivalent to placing the prior:

The output is insensitive to random perturbations of the input

Again, this is a form of implicit regularization.

Increasing noise, no change in label.

Example: Linear Regression with Noisy Inputs

Recall: least squares problem minimizes

$$R(w) = \frac{1}{2} \|Xw - y\|^2$$

Suppose we add noise to the inputs, which replaces

$$X \mapsto \tilde{X} = X + Z,$$

$$Z^{(i)} \sim N(0, \delta I)$$

where $Z^{(i)}$ denotes the i^{th} row of Z .

Then

$$\tilde{R}(w) = \frac{1}{2} \|\tilde{X}w - y\|^2 = \frac{1}{2} \|Xw - y\|^2 + \frac{1}{2} \|Zw\|^2 + (Zw)^T (Xw - y)$$

And so

$$\mathbb{E}_Z[\tilde{R}(w)] = R(w) + \frac{1}{2} \delta N \|w\|^2$$

That is, adding noise to the input here is equivalent to L^2 regularization.

This is also heuristically true in general.

Adding Noise to the Output

The rationale behind injecting noise to the output varies from application to application.

Examples:

- For classification problems, the given labels may have a small probability of being wrong
 - The labels / outputs could be randomly drawn according to some distribution
- In these cases, it may be advantageous to model such noise explicitly.

Label Smoothing

An oft-used example of output noise injection is label smoothing.

For a classification problem with a one-hot label, we can smooth the label by placing

$$1 \rightarrow 1 - \alpha$$

and

$$0 \rightarrow \frac{\alpha}{K - 1}$$

Example:

$$y = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \rightarrow y = \begin{pmatrix} \alpha/3 \\ 1 - \alpha \\ \alpha/3 \\ \alpha/3 \end{pmatrix}$$

Adding Noise to the Weights

Noise can also be added to the weights during training.

Consider one input-output pair (x, y) .

Suppose we have a neural network which makes the prediction

$$\hat{y} = f(x, \theta)$$

Now, we add a small perturbation $\delta\phi$ ($\delta \ll 1$) to θ ,

$$\phi \sim N(0, I)$$

Then the risk (averaged over ϕ) is

$$\begin{aligned} \mathbb{E}_\phi \tilde{R}(\theta) &= \frac{1}{2} \mathbb{E}_\phi (f(x, \theta + \delta\phi) - y)^2 \\ &= \frac{1}{2} (f(x, \theta) - y)^2 + \frac{m\delta^2}{2} \|\nabla_\theta f(x, \theta)\|^2 + \dots \end{aligned}$$

That is, this penalizes variations of the neural network predictions with respect to the weights.

Bootstrap Aggregating (Bagging)

- Bagging is a simple method for combining models.
- Train m models $\{f_1, \dots, f_m\}$ on random subsamples of the training data.
- Combine predictions from all models.

Prediction Aggregation

- Regression:

$$\bar{f}(x) = \frac{1}{m} \sum_{j=1}^m f_j(x)$$

- Classification:

$$\bar{f}(x) = \text{Mode}\{f_j(x) : j = 1, \dots, m\}$$

- Other voting schemes may also be used.

What Does Bagging Do?

- Consider a scalar model

$$f_i(x) = f^*(x) + \epsilon_i(x)$$

- Noise assumptions:
 - $\mathbb{E}[\epsilon_i(x)] = 0$
 - $\mathbb{E}[\epsilon_i(x)^2] = \sigma(x)^2$
 - $\mathbb{E}[\epsilon_i(x)\epsilon_j(x)] = 0$ for $i \neq j$
- Aggregate predictor

$$\bar{f}(x) = \frac{1}{m} \sum_{j=1}^m f_j(x)$$

Error Reduction

- Define errors

$$E(x) = \frac{1}{m} \sum_{j=1}^m \mathbb{E}(f_j(x) - f^*(x))^2$$

$$\bar{E}(x) = \mathbb{E}(\bar{f}(x) - f^*(x))^2$$

- Result:

$$\bar{E}(x) = \frac{1}{m} E(x)$$

- Bagging reduces variance by a factor of m .
- Key unrealistic assumption: errors are independent.

Dropout

- Storing many models in ensembles is expensive.
- Dropout approximates model ensembling efficiently.
- Similar idea to stochastic approximation in SGD.

$$\frac{1}{m} \sum_i f_i = \mathbb{E}_\gamma f_\gamma$$

$$\gamma \sim \text{Uniform}\{1, \dots, m\}$$

- Instead of computing the full sum, sample one model index γ .

Dropout in Neural Networks

- Randomly mask nodes/weights during training.

$$f(x; W, b, \mu) = \sigma(W(\mu \circ x) + b)$$

- μ is a binary mask applied element-wise.

Dropout Sampling

- During SGD, generate mask μ :
 - each entry is 1 with probability p
 - each entry is 0 with probability $1 - p$
- p is the keep probability.
- Expected mask:

$$\mathbb{E}[\mu] = p(1, 1, \dots, 1)$$

- During prediction use

$$W \leftarrow pW$$

- This is the **weight scaling inference rule**.

Remarks on Dropout

- Ensemble of models

$$\{f(x, \theta, \mu) : \mu_i \in \{0, 1\}\}$$

- Total models: 2^m
- Models share parameters θ .
- Training complexity does not increase significantly.
- Ensemble average approximated by

$$\mathbb{E}_\mu f(x, \theta, \mu)$$

- Implemented via weight scaling at inference.

Dropout for Linear Regression

- Model:

f(x,w,μ)=1pwT(μ∘x)

- Empirical risk averaged over masks

R~(w)=Eμ12||X[Dμ/p]w−y||2

- Dμ is a diagonal matrix with μ on the diagonal.
- Solving yields

R~(w)=12||Xw−y||2+12wTQ(X,p)w

- Dropout behaves like weighted L2 regularization.

Motivation: Deep Linear Network

- Consider a 1D deep linear network

y~(x;w)=xw1w2...wl

- Loss (square loss)

L(w)=12y~(x;w)2

- Gradient update

w'=w−ϵg

- New loss approximation

L(w')=L(w)−ϵ||g||2+O(ϵ2)

- Descent requires ϵ small enough so the first term dominates.

Deep Network Expansion

- After update:

y~(x;w')=x(w1−ϵg1)...(wl−ϵgl)

- Expanding yields higher order interaction terms.
- One O(ϵ2) term:

2ϵ2xg~(x,w)g1g2(w3w4...wl)

- Hard to keep small if the product w3w4...wl is large.

Lessons from the Example

- In deep neural networks, the choice of learning rate depends on the scale of weights and activations across layers.
- These scales change during training, making it difficult to use a constant learning rate.
- Idea: normalize values at each layer so their statistics are similar.
- This motivates Batch Normalization (BN).

Batch Normalization Layer

Let

H={h(1),h(2),...,h(B)}

be a batch of hidden activations.

BN(H;γ,β)(i)=γ∘(h(i)−μσ)+β

where

μ=1B∑jh(j)

σ=√1B∑j(h(j)−μ)2

- γ,β∈Rd
- ∘ denotes elementwise multiplication.

BN Layer Breakdown (Normalization)

First normalize the activations

h~(i)=h(i)−μσ

- Ensures mean 0

1B∑ih~(i)=0

- Ensures variance 1

1B∑i(h~j(i))2=1

- Hidden states are standardized within the batch.

Rescaling and Recentering

Normalization alone reduces expressive power:

- outputs always centered at 0
- scale is order 1

Introduce learnable parameters:

h^(i)=γ∘h~(i)+β

Using BN in Deep Networks

Standard layer:

Hk+1=ReLU(HkW+b)

With Batch Normalization:

H'k=HkW+b

H~k=BN(H'k;γ,β)

Hk+1=ReLU(H~k)

BN During Training vs Inference

During training:

h~(i)=h(i)−μσ

using batch statistics.

During inference:

- maintain moving averages of μ,σ during training
- use those averages during prediction

Effect of Batch Normalization

- Removes scaling and centers activations.
- Gradients propagate through normalization.
- Decouples layer statistics via parameters γ,β.

Additional Remarks

For convolutional networks:

- β,γ are defined per channel.
- If feature map size is

[#Batch,#Height,#Width,#Channel]

- then β,γ have dimension #Channel.

BN is typically applied between affine transform and nonlinearity:

ReLU(XW+b)→ReLU(BN(XW+b))

Bias can be absorbed:

BN(XW+b)=BN(XW)

with β↦β+b.

Data vs Generalization

Supervised learning setup:

- Data distribution (x,y)~μ
- Samples (x(i),y(i))

Empirical risk:

Recmp(N)(θ)=1N∑iL(y~(x(i),θ),y(i))

Population risk:

Rpop(θ)=Ex,y~μ[L(y~(x,θ),y)]

Goal: reduce generalization gap

|Recmp(N)(θ)−Rpop(θ)|

Learning theory often gives bounds:

|Recmp(N)(θ)−Rpop(θ)|≤Model ComplexityN

Ways to reduce the gap:

- reduce complexity (regularization)
- increase dataset size (data augmentation)

Data Augmentation

- Data augmentation increases the effective size of the dataset to reduce the generalization gap.
- Idea: perturb a datapoint to generate a new sample with the same label.

Augmentation and Invariance

Let the oracle function be

y=f*(x)

Apply a random augmentation

x↦g(x;ξ)

where ξ is a random variable.

Fundamental Principle

f*(g(x;ξ)) ≈ y = f*(x)

- We assume the oracle is invariant to transformation g(·;ξ).
- Data Augmentation as Prior Knowledge**
- Augmentation injects prior knowledge about invariances.
- Examples:
 - Random translations → classifier invariant to translations
 - Random rotations → classifier invariant to rotations
- Therefore augmentation must reflect real invariances.

Convergence vs Learning Rate

Recall properties of SGD:

- Computationally efficient
 - Introduces noise
- For a quadratic problem

ER(θk) ~ R(θ0)(1 - ε)^{2k} + ε(1 - (1 - ε)^{2k})

- First term: exponential convergence
 - Second term: O(ε) fluctuations
- Thus learning rate should decrease during training.

ε = εk

Choosing the Learning Rate

Theoretical convergence conditions:

Σ_{k=0}^∞ εk = ∞

Σ_{k=0}^∞ εk^2 < ∞

Example schedule

εk = a / (1 + bk)

- Called an O(k^{-1}) decay schedule.
- Often not used in learning because:
 - convergence is slow
 - objective is not purely optimization

Exponential Learning Rate Schedule

Instead we commonly use exponential decay.

Parameters:

- Initial learning rate: ε0
- Decay step: k0
- Decay factor: γ

Schedule:

εk = ε0γ^{⌊k/k0⌋}

Robustness of Classifiers

Suppose classifier f̂ correctly predicts

f̂(x) = y

We call it robust if

f̂(x') = y

for all small perturbations

||x' - x|| ≪ 1

Finding Adversarial Examples

One approach is to add noise.

A stronger approach:

- find the worst perturbation within radius δ
- Given trained parameter θ̂

x' = arg max_{z, ||z-x|| ≤ δ} L(y(z; θ̂), y)

Adversarial Training

Reduce adversarial effects via a min-max problem

min_{θ̂} max_{||x-z|| ≤ δ} L(y(z; θ), y)

For dataset size N

min_{θ̂} 1/N Σ_{i=1}^N max_{||xi-z_i|| ≤ δ} L(ŷ(zi; θ), yi)

- Model learns parameters robust to worst-case perturbations.

Example: Effect of Adversarial Training

Consider a 1D example:

- Model: ŷ(x; θ) = θx
- Data: x = 1, y = 0

Loss Function

L(y, ŷ) = { 1/2 (ŷ - y - Sign(ŷ - y))^2 | ŷ - y| > 1, 0 | ŷ - y| ≤ 1 }

Standard Gradient Descent

Assume training starts from

θ0 > 1

Then GD converges to

θ∞^{(GD)} = 1

with

L(ŷ(x; θ∞^{(GD)}), y) = 0

- A small perturbation to the left does not increase loss.
- A perturbation to the right increases loss.

Adversarial Loss

Define the inner maximization

Ladv(θ) = max_{z: ||x-z|| ≤ δ} L(ŷ(z; θ), y)

For this model:

Ladv(θ) = max_{z: |1-z| ≤ δ} 1/2 (θz - Sign(θz))^2 1_{|θz| ≥ 1}

This yields

Ladv(θ) = 1/2 (θ(1+δ) - 1)^2 1_{|θ(1+δ)| ≥ 1}

- For the GD solution:

Ladv(θ∞^{(GD)}) = δ^2 / 2 > 0

- Adversarial loss becomes zero only when

|θ| ≤ 1 / (1 + δ)

Adversarial Training Algorithm

FGSM (Fast Gradient Sign Method)

Hyperparameters:

- δ : adversarial perturbation size
 - J : number of adversarial steps
 - ε1, ε2 : learning rates
- For iteration k = 0, 1, ...
- Initialize adversarial example

z0 = x

- Generate adversarial perturbation
- For j = 0, 1, ..., J - 1

zj+1 = zj + ε2 Sign(∇z L(ŷ(zj; θk), y))

- Update model parameters

θk+1 = θk - ε1 ∇θ L(ŷ(zj; θk), y)

- Train using adversarially perturbed samples.

Lecture 8

Unsupervised Learning

- Unsupervised learning is used when we do not have labels.
- Goal: learn task-agnostic patterns directly from the inputs.

The PCA Algorithm

- Data matrix:

X ∈ ℝ^{N × d}

- Covariance matrix:

S = 1/N X^T X

- Eigenvalue decomposition:

S = UΛU^T

- Columns of U are principal components
- Λ = diag(λ1, ..., λd) are explained variances
- Eigenvalues are sorted in descending order
- Principal component scores:

Z = XU

- Reconstruction:

X = ZU^T

PCA as a Compression Algorithm

- PCA provides a natural way to compress data.
- Keep only the first m components:

$$Z_m = XU_m, \quad U_m \in \mathbb{R}^{d \times m}$$

- Z_m is an m -dimensional compressed representation of X .
- Reconstruction:

$$X \approx X_m = Z_m U_m^T$$

Reconstruction Error

$$X_m = XU_m U_m^T$$

For each data point:

$$x^{(i)} = \sum_{j=1}^d (u_j^T x^{(i)}) u_j \quad \text{and} \quad x_m^{(i)} = \sum_{j=1}^m (u_j^T x^{(i)}) u_j$$

Thus:

$$x^{(i)} - x_m^{(i)} = \sum_{j=m+1}^d (u_j^T x^{(i)}) u_j$$

Averaged error:

$$\frac{1}{N} \sum_{i=1}^N \|x^{(i)} - x_m^{(i)}\|^2 = \sum_{j=m+1}^d \lambda_j$$

Autoencoders

- Autoencoders are nonlinear counterparts of PCA.
- PCA:

$$z = U_m^T x, \quad x' = U_m z$$

- Autoencoder:

$$z = f(x), \quad x' = g(z)$$

Neural Network Autoencoders

- Neural networks parameterize encoder f and decoder g .
- Functions:

$$f: \mathbb{R}^d \rightarrow \mathbb{R}^m, \quad g: \mathbb{R}^m \rightarrow \mathbb{R}^d$$

- $\mathbb{R}^m$ is called the latent space.
- $z = f(x)$ is the latent representation (latent variable/state).

Learning Autoencoders

- Training objective:

$$L(x, x') = L(x, g(f(x)))$$

- Equivalent to supervised learning where target = input.

Undercomplete Autoencoders

- Latent dimension is small ($m \ll d$).
- Used for compression and feature extraction.
- If f, g are linear and loss is MSE, equivalent to PCA.

Regularized Autoencoders

- Risk: model memorizes input-output mapping.
- Causes: Latent dimension too large, networks too complex.
- Solution: add regularization:

$$L(x, g(f(x))) + \Omega(x, f, g)$$

- Sparse autoencoder:

$$\Omega = \alpha \|f(x)\|_1$$

- Contractive autoencoder:

$$\Omega = \alpha \|\nabla f(x)\|_2^2$$

Denoising Autoencoders

- Autoencoders can also be used for denoising.
- Instead of minimizing $L(x, g(f(x)))$, we minimize $L(x, g(f(\tilde{x})))$
- $\tilde{x}$ is a noise-perturbed version of x .
- After training, the model removes noise:

$$x \approx g(f(\tilde{x}))$$

Seq-to-Seq (Auto-)Encoders

- Autoencoders can be applied to sequence data.
- Encoder f and decoder g are recurrent networks.
- Can handle variable-length inputs.

Semi-supervised Learning

- In practice, unlabeled data is much more abundant than labeled data.
- Unlabeled data is easy to collect.
- Labeled data requires human annotation.

Data for Semi-supervised Learning

Supervised dataset:

$$\mathcal{D} = \{x^{(i)}, y^{(i)} : i = 1, \dots, N\}$$

Semi-supervised datasets:

$$\mathcal{D}_L = \{x^{(i)}, y^{(i)} : i = 1, \dots, N_L\}$$

$$\mathcal{D}_U = \{\tilde{x}^{(i)} : i = 1, \dots, N_U\}$$

- $\mathcal{D}_L$: labeled dataset
- $\mathcal{D}_U$: unlabeled dataset
- Typically $N_U \gg N_L$

Two Types of Semi-supervised Problems

- **Transductive:** label the unlabeled dataset $\mathcal{D}_U$
- **Inductive:** learn f^* to generalize to unseen data

How Can Unlabeled Data Help?

- Unlabeled data can improve the learned decision boundary.
- Helps align decision boundaries with the data distribution.

Underlying Assumptions

- **Continuity assumption:** nearby points have similar labels
- **Cluster assumption:** data form clusters with consistent labels
- **Manifold assumption:** data lie on low-dimensional manifolds

Self-training (Simplest Heuristic)

- Use a model trained on $\mathcal{D}_L$ to label $\mathcal{D}_U$.

Algorithm:

1. Train $\hat{f}$ on $\mathcal{D}_L$
2. For $\tilde{x} \in \mathcal{D}_U$, predict $y = \hat{f}(\tilde{x})$
3. Add $(\tilde{x}, y)$ to $\mathcal{D}_L$ and repeat

Variations in Self-training

- Add only most confident predictions:

$$(\tilde{x}^{(i)}, \hat{f}(\tilde{x}^{(i)}))$$

- Add all predictions
- Weight samples by confidence

Pros and Cons of Self-training

Advantages:

- Simple to implement
- General meta-algorithm

Disadvantages:

- Early mistakes may reinforce themselves
- Limited theoretical guarantees

Label Propagation

- Another commonly used method for semi-supervised learning is label propagation.
- The primary idea is based on the clustering and manifold assumptions.

Basic Label Propagation Algorithm

1. Represent samples as a graph (edges encode similarity).
2. Assign to each unlabeled sample the majority label of its neighbors (random order).
3. If all labels are stable (match neighbor majority), stop; otherwise repeat.

Visualization Insight

- Labeled points propagate labels to nearby unlabeled points.
- Confidence increases with consistency of neighbors.
- Decision boundaries align with data structure.

Multi-Task Learning

- Learn multiple tasks simultaneously.
- Or learn shared representations across tasks.
- Tasks can be supervised, unsupervised, or mixed.

Two key approaches:

- Transfer learning
- Domain adaptation

Manifold Assumption

- Relevant features across tasks share structure.
- Data lie approximately on a low-dimensional manifold.

Input data lie on a low-dimensional manifold

Transfer Learning

- Data across tasks are similar, but tasks differ.
- Shared features can be reused across tasks.

Key idea:

Similar data $\Rightarrow$ shared feature representations

Practical approaches:

- Share hidden layers
- Fine-tune pre-trained networks
- Use unsupervised pre-training (e.g. autoencoders)

Sharing Hidden States

- For tasks $y^{(1)}$ and $y^{(2)}$, use shared representation $h^{(\text{shared})}$.
- Both tasks influence the learned representation.

Multiple Inputs, Common Output

- Different input modalities can map to a shared representation.
- Shared representation feeds a common output.

Using Pre-trained Networks

- Use a model trained on task 1 to initialize task 2.
- This provides a warm start.
- Modify input/output layers if semantics differ.

Domain Adaptation

- Tasks P_1 and P_2 are identical.
- Input distributions differ.
- Goal: adapt model across domains.

Domain Adaptation Algorithms

- There are many domain adaptation algorithms.
- Some are closer to standard semi-supervised learning than others.

Examples

- Label propagation using a small set of labeled samples in the target domain.
- Learn a common representation space across domains (e.g. autoencoders, adversarial methods).
- Learn transport maps to match distributions.

Lecture 9

Discriminative vs Generative Models

- Discriminative modelling: $f^*(x)$
- Generative modelling: $p^*(x)$

Modelling Distributions

- Two main approaches to model distributions.

Density Estimation

- Given: $\mathcal{D} = \{x^{(1)}, \dots, x^{(N)}\}$, $x^{(i)} \sim p^*$
- Goal: find $\hat{p} \approx p^*$

Generative Models

- Given: $\mathcal{D} = \{x^{(1)}, \dots, x^{(N)}\}$, $x^{(i)} \sim p^*$
- Goal: sample $\tilde{x} \sim p^*$ (approximately)

Differences

- With $\hat{p}$, sampling may be difficult.

Gaussian Mixture Models

- $p_\theta(x) = \sum_j \alpha_j p_{\mu_j, \Sigma_j}(x)$
- $\alpha_j \geq 0$, $\sum_j \alpha_j = 1$
- $p_{\mu, \Sigma}(x)$: Gaussian PDF with mean μ and covariance Σ

Sampling from GMM

- Sample index $i \sim \{\alpha_j\}$
- Sample $x \sim p_{\mu_i, \Sigma_i}$

Limitations of GMM

- Poor scaling to high dimensions
- Number of components is hard to tune
- Cannot easily incorporate domain knowledge

Deep Learning Based Generative Models

- Autoencoder: encoder $\rightarrow$ latent $\rightarrow$ decoder
- Idea: generate in latent space, then decode

Issue with Autoencoders

- Latent space is not continuous
- Nearby latent points may not decode smoothly

Maximum Likelihood Estimation

- Model $p_\theta(x)$ with data $\{x^{(i)}\}$
- $\hat{\theta} = \arg \max_\theta \prod_i p_\theta(x^{(i)})$
- Equivalent: $\hat{\theta} = \arg \max_\theta \sum_i \log p_\theta(x^{(i)})$

Gaussian MLE (1D)

- $p_\theta(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$
- $\hat{\mu} = \frac{1}{N} \sum_i x^{(i)}$
- $\hat{\sigma}^2 = \frac{1}{N} \sum_i (x^{(i)} - \hat{\mu})^2$

Sampling with MLE

- After fitting: $x' \sim p_{\hat{\theta}}(x)$

Limitations of MLE

- Simple models may not fit data well
- Complex models are hard to optimize/sample

Latent Generative Models

- $p_\theta(x) = \int p_\theta(x|z)p_\theta(z) dz$
- z : latent variable
- $p_\theta(z)$: prior
- $p_\theta(x|z)$: conditional/generative model

Example: Latent Generative Model

Let

- z be a uniform random variable on $[a, b]$
- x be conditionally normal with mean μ and variance z^2

Then

$$p_\theta(z) = \frac{1}{b-a} \mathbf{1}_{a \leq z \leq b}$$

$$p_\theta(x | z) = \frac{1}{\sqrt{2\pi z^2}} \exp\left(-\frac{(x-\mu)^2}{2z^2}\right)$$

with $\theta = (a, b, \mu)$

Direct vs Latent Approach

Given a data point x

Direct Approach

- Learn $p_\theta(x)$
- Sample $x' \sim p_\theta(x)$

Latent Variable Approach

- Learn posterior $p_\theta(z | x)$
- Learn generative model $p_\theta(x | z)$
- Sample $z \sim p_\theta(z | x)$, then $x' \sim p_\theta(x | z)$

Posterior Model

By Bayes theorem,

$$p_\theta(z | x) = \frac{p_\theta(x | z)p_\theta(z)}{\int p_\theta(x, z') dz'}$$

- The denominator is computationally intractable
- Instead, introduce an approximate posterior

$$q_\phi(z | x) \approx p_\theta(z | x)$$

Variational Autoencoders

- Parameters ϕ are learned to make the approximation accurate
- Latent-variable generative models resemble autoencoders
- Called **variational autoencoders (VAEs)**
- “Variational” refers to approximating $p_\theta(z | x)$ by $q_\phi(z | x)$

Key Questions

- How to parameterize $p_\theta(x | z)$ and $q_\phi(z | x)$?
- How to train θ and ϕ ?

Training AEs vs VAEs

Autoencoders

$$\min_{\theta, \phi} L(x, x') = L(x, g_\theta(f_\phi(x)))$$

- Input equals target

VAEs

- x' and x are not required to match exactly
- We train distributions q_ϕ and p_θ

KL Divergence

Given densities $p(x)$ and $q(x)$,

$$D_{KL}(p \| q) = \int p(x) \log \frac{p(x)}{q(x)} dx = \mathbb{E}_{p(x)} \left[\log \frac{p(x)}{q(x)} \right]$$

Properties

- $D_{KL}(p \| q) \geq 0$, equality iff $p = q$
- Convex in p and q
- Not symmetric

KL Divergence (Gaussian Example)

For 1D Gaussians,

$$D_{KL}(p_{\theta_1} \| p_{\theta_2}) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2}{2\sigma_2^2} - \frac{1}{2} + \frac{(\mu_1 - \mu_2)^2}{2\sigma_2^2}$$

- Non-negative
- Zero iff $\mu_1 = \mu_2$ and $\sigma_1 = \sigma_2$
- Not symmetric

Monte-Carlo Estimation

For $x \sim p(x)$,

$$\mathbb{E}[f(x)] = \int f(x)p(x)dx$$

Approximate with samples:

$$\mathbb{E}[f(x)] \approx \frac{1}{N} \sum_{i=1}^N f(x^{(i)}), \quad x^{(i)} \sim p$$

- Becomes exact as $N \rightarrow \infty$

Evidence Lower Bound (ELBO)

Let us apply MLE to the latent model

$$\log p_\theta(x) = \log \int p_\theta(x | z)p_\theta(z) dz$$

- The integral is intractable

We can show

$$\log p_\theta(x) = \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)} \right] + D_{KL}(q_\phi(z | x) \| p_\theta(z | x))$$

- First term: ELBO $L(x; \theta, \phi)$
- Second term: KL divergence

Optimizing the ELBO

- KL divergence is non-negative

$$\log p_\theta(x) = L(x; \theta, \phi) + K(x; \theta, \phi) \geq L(x; \theta, \phi)$$

- L is a lower bound of log-likelihood

$$\max_\theta \log p_\theta(x) \Rightarrow \max_{\theta, \phi} L(x; \theta, \phi)$$

- Can optimize without exact posterior

Multiple Datapoints

$$L(\mathcal{D}; \theta, \phi) = \frac{1}{N} \sum_{i=1}^N L(x^{(i)}; \theta, \phi)$$

$$= \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{q_\phi(z|x^{(i)})} \left[\log \frac{p_\theta(x^{(i)}, z)}{q_\phi(z | x^{(i)})} \right]$$

- Optimize via (stochastic) gradient descent, need $\nabla_\theta L$ and $\nabla_\phi L$

Gradient w.r.t. θ

$$\nabla_\theta L = \mathbb{E}_{q_\phi(z|x)} [\nabla_\theta \log p_\theta(x, z)]$$

- Computable via Monte-Carlo if we can sample $z \sim q_\phi(z | x)$

Gradient w.r.t. ϕ

$$\nabla_\phi L = \nabla_\phi \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)} \right]$$

- More involved, need to enable Monte-Carlo + backprop

Reparameterization Trick

$$z \sim q_\phi(z | x) \Rightarrow z = g(u, \phi, x)$$

- $u \sim p_0(u)$ independent of ϕ
- g deterministic

Example: Gaussian

$$z \sim \mathcal{N}(\mu + x, C) \quad \Rightarrow \quad z = \mu + x + C^{1/2}u, \quad u \sim \mathcal{N}(0, I)$$

Why Reparameterize?

$$\mathbb{E}_{z \sim q_\phi} [f(z)] = \mathbb{E}_{u \sim p_0} [f(g(u, \phi, x))]$$

$$\nabla_\phi \mathbb{E}_{z \sim q_\phi} [f(z)] = \mathbb{E}_{u \sim p_0} [\nabla_\phi f(g(u, \phi, x))]$$

- Enables Monte-Carlo gradient estimation

Final Objective

$$L(x; \theta, \phi) = \mathbb{E}_{p_0(u)} [\log p_\theta(x, g(u, \phi, x)) - \log q_\phi(g(u, \phi, x) | x)]$$

- Gradients computable via Monte-Carlo
- Neural Network Parameterization**
- Choose simple prior $p_\theta(z)$ (e.g. iid Gaussian)
 - Parameterize $p_\theta(x | z)$ and $q_\phi(z | x)$ with neural networks
 - Must be:
 - Easy to evaluate
 - Differentiable
 - Easy to sample from

Representing $p_\theta(x | z)$

Choice depends on data type. For example:

Suppose $x \in \{0, 1\}^d$ (binary data). Then a common choice is the *factorized Bernoulli model*.

Recall: Let x be a scalar Bernoulli random variable, then

$$x = \begin{cases} 1 & \text{with prob } s \\ 0 & \text{with prob } 1 - s \end{cases}$$

Here $s \in [0, 1]$ is the probability of successful outcome. The distribution function then depends on s , and is given by

$$p_s(x) = s^x (1 - s)^{1-x}$$

In general, we may represent $p_\theta(x | z)$ as a factorized Bernoulli model with the success probability vector s being a deterministic function of the latent variable z .

$$s = \text{DecodingNN}(z; \theta), \quad s \in [0, 1]^d$$

$$p_\theta(x | z) = \prod_j s_j^{x_j} (1 - s_j)^{1-x_j}$$

$$\log p_\theta(x | z) = \sum_j x_j \log s_j + (1 - x_j) \log(1 - s_j)$$

The last term is just the negative of the binary cross-entropy loss.

Representing $q_\phi(z | x)$

Similarly, we can pick the simplest case of factorized Gaussian model for the encoding distribution.

$$(\mu, \log \sigma) = \text{EncodingNN}(x; \phi)$$

$$q_\phi(z | x) = \prod_j (2\pi\sigma_j^2)^{-1/2} \exp\left(-\frac{1}{2\sigma_j^2}(z_j - \mu_j)^2\right)$$

Under the reparameterization trick, we have

$$z = g(u, \phi, x) = \mu + \sigma \circ u, \quad u \sim p_0 = \mathcal{N}(0, I)$$

Giving

$$\log q_\phi(z | x) = \sum_j -\frac{1}{2} \log(2\pi\sigma_j^2) - \frac{1}{2}u_j^2$$

Simplifying the ELBO Loss

Under the current choices, we can dramatically simplify the ELBO loss

$$\begin{aligned} \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)} \right] &= \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x | z)p_\theta(z)}{q_\phi(z | x)} \right] \\ &= \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z) + \log p_\theta(z) - \log q_\phi(z | x)] \\ &= \mathbb{E}_{p_0(u)} [\log p_\theta(x | g(u, \phi, x)) + \log p_\theta(g(u, \phi, x)) - \log q_\phi(g(u, \phi, x) | x)] \end{aligned}$$

We have

$$\log p_\theta(x | z) = \sum_j x_j \log s_j + (1 - x_j) \log(1 - s_j)$$

$$\log p_\theta(z) = -\frac{1}{2} \sum_j [z_j^2 + \log(2\pi)]$$

$$\log q_\phi(z | x) = -\frac{1}{2} \sum_j [u_j^2 + \log(2\pi) + 2 \log(\sigma_j)]$$

The only term that requires further attention is

$$\log p_\theta(z) = -\frac{1}{2} \sum_j [z_j^2 + \log(2\pi)]$$

Since we have to get rid of z in order to let the gradient flow through the reparameterization trick, we can show that (dropping constants)

$$\mathbb{E}_{p_0(u)} [\log p_\theta(z)] = \mathbb{E}_{u \sim \mathcal{N}(0, I)} \left[-\frac{1}{2} \sum_j (\mu_j + \sigma_j u_j)^2 \right] = -\frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2)$$

Summary of VAE (Basic Form)

Input dimension = d . Latent dimension = m .

Encoder

$$(y_1, y_2) = \text{EncodingNN}(x; \phi)$$

$$z = y_1 + e^{y_2} \circ u, \quad u \sim \mathcal{N}(0, I), \quad (u \in \mathbb{R}^m)$$

Decoder

$$s = \text{DecodingNN}(z; \theta), \quad (s \in \mathbb{R}^d)$$

Inference: $x' \sim \text{Bernoulli}(s)$

Loss

$$-L(x; \theta, \phi) = \text{BCE}(x, s) + \frac{1}{2} \|y_1\|^2 + \frac{1}{2} \|e^{y_2}\|^2 - \sum_j y_{2,j}$$

Some Remarks

- The loss is usually split into two parts

$$-L(x; \theta, \phi) = \text{BCE}(x, s) + \left[\frac{1}{2} \|y_1\|^2 + \frac{1}{2} \|e^{y_2}\|^2 - \sum_j y_{2,j} \right]$$

$$-L = -\mathbb{E}_{q_\phi(z|x)} \log p_\theta(x | z) + \mathbb{E}_{q_\phi(z|x)} \log \frac{q_\phi(z | x)}{p_\theta(z)} = \text{BCE}(x, s) + D_{KL}(q_\phi(z | x) \| p_\theta(z))$$

- The multiple-sample loss sums over all single sample losses

Lecture 10

Representing $p_\theta(x|z)$

- Choice depends on data type. For example: Suppose $x \in \{0, 1\}^d$ (binary data). Then a common choice is the *factorized Bernoulli model*.
- Recall: Let x be a scalar Bernoulli random variable, then

$$x = \begin{cases} 1 & \text{with prob } s \\ 0 & \text{with prob } 1 - s \end{cases}$$

- Here $s \in [0, 1]$ is the probability of successful outcome.
- The distribution function then depends on s , and is given by

$$p_s(x) = s^x (1 - s)^{1-x}$$

Representing $p_\theta(x|z)$

- In general, we may represent $p_\theta(x|z)$ as factorized Bernoulli model with the success probability vector s being a deterministic function of the latent variable z .

$$s = \text{DecodingNN}(z; \theta), \quad s \in [0, 1]^d$$

$$p_\theta(x|z) = \prod_j s_j^{x_j} (1 - s_j)^{1-x_j}$$

$$\log p_\theta(x|z) = \sum_j x_j \log s_j + (1 - x_j) \log(1 - s_j)$$

- The last term is just the negative of the binary cross-entropy loss.
- Representing $q_\phi(z|x)$**

- Similarly, we can pick the simplest case of factorized Gaussian model for the encoding distribution.

$$(\mu, \log \sigma) = \text{EncodingNN}(x; \phi)$$

- and

$$q_{\phi}(z|x) = \prod_j (2\pi\sigma_j^2)^{-1/2} \exp\left(-\frac{1}{2\sigma_j^2}(z_j - \mu_j)^2\right)$$

- Under the reparameterization trick, we have

$$z = g(u, \phi, x) = \mu + \sigma \circ u, \quad u \sim \mathcal{N}(0, I)$$

- Giving

$$\log q_{\phi}(z|x) = \sum_j -\frac{1}{2} \log(2\pi\sigma_j^2) - \frac{1}{2} u_j^2$$

Simplifying the ELBO Loss

- Under the current choices, we can dramatically simplify the ELBO loss.

$$\begin{aligned} \mathbb{E}_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} \right] &= \mathbb{E}_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x|z)p_{\theta}(z)}{q_{\phi}(z|x)} \right] \\ &= \mathbb{E}_{q_{\phi}(z|x)} \left[\log p_{\theta}(x|z) + \log p_{\theta}(z) - \log q_{\phi}(z|x) \right] \\ &= \mathbb{E}_{p_0(u)} \left[\log p_{\theta}(x|g(u, \phi, x)) + \log p_{\theta}(g(u, \phi, x)) - \log q_{\phi}(g(u, \phi, x)|x) \right] \\ \log p_{\theta}(x|z) &= \sum_j x_j \log s_j + (1 - x_j) \log(1 - s_j) \\ \log p_{\theta}(z) &= -\frac{1}{2} \sum_j [z_j^2 + \log(2\pi)] \\ \log q_{\phi}(z|x) &= -\frac{1}{2} \sum_j [u_j^2 + \log(2\pi) + 2 \log(\sigma_j)] \end{aligned}$$

Simplifying Further

- The only term that requires further attention is

$$\log p_{\theta}(z) = -\frac{1}{2} \sum_j [z_j^2 + \log(2\pi)]$$

- Since we have to get rid of z to let gradients flow:

$$\begin{aligned} \mathbb{E}_{p_0(u)} [\log p_{\theta}(z)] &= \mathbb{E}_{u \sim \mathcal{N}(0, I)} \left[-\frac{1}{2} \sum_j (\mu_j + \sigma_j u_j)^2 \right] \\ &= -\frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2) \end{aligned}$$

Summary of VAE (Basic Form)

- Input dimension = d . Latent dimension = m .
- Encoder:

$$(y_1, y_2) = \text{EncodingNN}(x; \phi)$$

$$z = y_1 + e^{y_2} \circ u, \quad u \sim \mathcal{N}(0, I)$$

- Decoder:

$$s = \text{DecodingNN}(z; \theta)$$

$$x' \sim \text{Bernoulli}(s)$$

- Loss:

$$-L(x; \theta, \phi) = \text{BCE}(x, s) + \frac{1}{2} \|y_1\|^2 + \frac{1}{2} \|e^{y_2}\|^2 - \sum_j y_{2,j}$$

Some Remarks

- The loss is usually split into two parts

$$-L(x; \theta, \phi) = \text{BCE}(x, s) + \left[\frac{1}{2} \|y_1\|^2 + \frac{1}{2} \|e^{y_2}\|^2 - \sum_j y_{2,j} \right]$$

- Why this name? Observe

$$\begin{aligned} -L &= -\mathbb{E}_{q_{\phi}(z|x)} \log p_{\theta}(x|z) + \mathbb{E}_{q_{\phi}(z|x)} \log \left(\frac{q_{\phi}(z|x)}{p_{\theta}(z)} \right) \\ &= \text{BCE}(x, s) + D_{KL}(q_{\phi}(z|x) \| p_{\theta}(z)) \end{aligned}$$

- The multiple-sample loss sums over all single sample losses.

Adversaries for Generative Models

- We make use of adversaries to generate outputs. This is known as Generative Adversarial Networks (GAN).
- Generator $\rightarrow$ Samples
- Discriminator (adversary) detects generated samples

Mathematical Formulation of the Training Problem

- Latent variable (n -dimensional)

$$z \sim p_0 = \mathcal{N}(0, I_n)$$

- Training data

$$\{x^{(i)} \in \mathbb{R}^d : i = 1, \dots, N, \quad x^{(i)} \sim p^*\}$$

- Generator:

$$G_{\theta} : \mathbb{R}^n \rightarrow \mathbb{R}^d$$

- Discriminator:

$$D_{\phi} : \mathbb{R}^d \rightarrow [0, 1]$$

- where $D_{\phi}(x)$ is the predicted probability of x being a real sample.

Mathematical Formulation of The Training Problem

- Discriminator's cost function is to minimize the cross-entropy loss for real vs generated samples.
- Recall, given true binary labels $y \in \{0, 1\}$, the (binary) cross-entropy loss is

$$-y \log D_{\phi}(x) - (1 - y) \log(1 - D_{\phi}(x))$$

- If $y = 1$ (real sample), then the average loss is

$$\mathbb{E}_{x \sim p^*} [-\log D_{\phi}(x)]$$

- If $y = 0$ (generated sample), then the average loss is

$$\mathbb{E}_{z \sim p_0} [-\log(1 - D_{\phi}(G_{\theta}(z)))]$$

- Hence, the total (negative) loss the discriminator should maximize is

$$L(\theta, \phi) = \mathbb{E}_{x \sim p^*} [\log D_{\phi}(x)] + \mathbb{E}_{z \sim p_0} [\log(1 - D_{\phi}(G_{\theta}(z)))]$$

- Remark: We assume equal amounts of real and generated samples are fed to the discriminator.

Min-Max Problem

- Just like adversarial training, the adversary maximizes the loss, and training minimizes it:

$$\min_{\theta} \max_{\phi} L(\theta, \phi)$$

- Writing this out (and negating), we have

$$\min_{\theta} \max_{\phi} \mathbb{E}_{x \sim p^*} [\log D_{\phi}(x)] + \mathbb{E}_{z \sim p_0} [\log(1 - D_{\phi}(G_{\theta}(z)))]$$

- This is called:
 - Min-max (minimax) formulation
 - Saddle-point formulation
 - Zero-sum game between D_{ϕ} and G_{θ}

Relationship with Game Theory

- Interpreting as a two-player zero-sum game, we ask where the Nash equilibrium is.
- A Nash equilibrium is where neither player has incentive to change its strategy.
- A Nash equilibrium:
 - The distribution of $G_{\theta}(z)$ equals p^*
 - $D_{\phi}(x) = \frac{1}{2}$ for all x

Relationship with KL Divergence

- Ignoring parameterizations, write:

$$\min_G \max_D \mathbb{E}_{x \sim p^*} [\log D(x)] + \mathbb{E}_{z \sim p_0} [\log(1 - D(G(z)))]$$

- Denote by p_G the distribution of $G(z)$. For fixed G , the optimal D is

$$D_G^*(x) = \frac{p^*(x)}{p^*(x) + p_G(x)}$$

- Thus, the minimax problem becomes

$$\min_G D_{KL} \left(p^* \left\| \frac{p^* + p_G}{2} \right\| \right) + D_{KL} \left(p_G \left\| \frac{p^* + p_G}{2} \right\| \right) - 2 \log 2$$

KL Divergence and JS Divergence

- Recall the KL divergence

$$D_{KL}(p \| q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

- The Jensen-Shannon (JS) divergence is

$$D_{JS}(p\|q) = \frac{1}{2}D_{KL}\left(p\left\|\frac{p+q}{2}\right.\right) + \frac{1}{2}D_{KL}\left(q\left\|\frac{p+q}{2}\right.\right)$$

- Properties:
 - Non-negative and equals 0 iff $p = q$
 - Symmetric (unlike KL)
 - $\sqrt{D_{JS}}$ is a metric

Training GAN from the Divergence Point of View

- The GAN training problem becomes

$$\min_G D_{JS}(p_G\|p^*)$$

- That is, we minimize the JS divergence between generated and true distributions.

The Basic Training Algorithm for GAN

- Repeat:
 - For k steps (train discriminator):
 - * Sample m noise samples $\{z^{(1)}, \dots, z^{(m)}\} \sim p_0$
 - * Sample m real samples $\{x^{(1)}, \dots, x^{(m)}\} \in \mathcal{D}$
 - * Update

$$\phi \leftarrow \phi + \epsilon_1 \nabla_{\phi} \frac{1}{m} \sum_{i=1}^m \left[\log D_{\phi}(x^{(i)}) + \log(1 - D_{\phi}(G_{\theta}(z^{(i)}))) \right]$$

- Train generator:
 - * Sample $\{z^{(i)}\} \sim p_0$
 - * Update

$$\theta \leftarrow \theta - \epsilon_2 \nabla_{\theta} \frac{1}{m} \sum_{i=1}^m \log(1 - D_{\phi}(G_{\theta}(z^{(i)})))$$

Problem I: Vanishing Gradients

- Loss function

$$L(\theta, \phi) = \mathbb{E}_{x \sim p^*} [\log D_{\phi}(x)] + \mathbb{E}_{z \sim p_0} [\log(1 - D_{\phi}(G_{\theta}(z)))]$$

- Gradient wrt θ

$$\nabla_{\theta} L(\theta, \phi) = \nabla_{\theta} \mathbb{E}_{z \sim p_0} [\log(1 - D_{\phi}(G_{\theta}(z)))]$$

- If σ is sigmoid:
 - $\nabla_a \log(1 - \sigma(a)) = -\sigma(a)$
 - Hence

$$\nabla_{\theta} L \propto \mathbb{E}_{z \sim p_0} D_{\phi}(G_{\theta}(z))$$

- We have shown

$$\nabla_{\theta} L \propto \mathbb{E}_{z \sim p_0} D_{\phi}(G_{\theta}(z)) \approx 0 \quad \text{if } D_{\phi} \approx 0$$

- This means:
 - Early in training, G_{θ} produces poor samples
 - D_{ϕ} rejects them with high confidence
 - Gradients vanish $\Rightarrow$ generator stops learning

A Heuristic to Solve the Gradient Vanishing Problem

- Replace the generator objective:

$$\nabla_{\theta} \mathbb{E}_{z \sim p_0} [\log(1 - D_{\phi}(G_{\theta}(z)))]$$

- With:

$$\nabla_{\theta} \mathbb{E}_{z \sim p_0} [-\log D_{\phi}(G_{\theta}(z))]$$

Problem II: Non-convergence

- Besides the gradient vanishing problem, in general GD is not designed to solve minimax problems.
- Consider solving

$$\min_x \max_y L(x, y) = xy$$

- The solutions are $x = 0, y \in \mathbb{R}$.
- However, GD by alternating inner and outer loop will not find this solution!

Problem III: Mode Collapse

- Mode collapse refers to the phenomenon that the generated samples are not varied and becomes very similar to a training sample.

Using Mini-Batch Statistics

- Mode collapse can be detected, since generated samples are of small variance.
- To combat this, we can use *Mini-batch GAN*.
 - Generation happens in mini-batches, with batch statistics (e.g. variance, pair-wise L^2 norms) fed into discriminator.
 - Discriminator can tag an entire batch of examples as false if the variation is low.
 - Hence, generator is forced to produce varied samples.

Wasserstein GANs

The Distance Viewpoint

- Let us return to the viewpoint of training GANs as minimizing “distances” between distributions.
- Recall that training GAN can be formulated as

$$\min_G D_{JS}(p_G\|p^*)$$

$$= \min_G \frac{1}{2} D_{KL}\left(p_G\left\|\frac{p^* + p_G}{2}\right.\right) + \frac{1}{2} D_{KL}\left(p^*\left\|\frac{p^* + p_G}{2}\right.\right)$$

What's wrong with KL/JS divergence?

- KL/JS divergences don't play nice when the distributions are *singular*.

Example: Singular KL Divergence

- Let

$$p(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} \quad \text{and} \quad q(x) = \frac{1}{2}\mathbb{I}_{x=1} + \frac{1}{2}\mathbb{I}_{x=-1}$$

- Then

$$D_{KL}(p\|q) = \int p(x) \log\left(\frac{p(x)}{q(x)}\right) dx$$

- is undefined since for all $x \neq \pm 1$, $q(x) = 0$ but $p(x) \neq 0$.
- In general, $D_{KL}(p\|q)$ is finite if and only if p is *absolutely continuous* with respect to q .
- Meaning that for any subset A ,

$$\int_A q(x) dx = 0 \Rightarrow \int_A p(x) dx = 0$$

- Otherwise, the D_{KL} is infinite, and we say that p is *singular* with respect to q .

Continuity and Distance

- As in VAE and GAN, our primary idea of representing a distribution is to:
 - Sample latent $z \sim p_0$ (simple distribution)
 - Generate samples $G(z)$ with desired distribution p_G by adjusting G
 - With parameterization: $G(z) = G(z; \theta)$ so $p_G = p_{\theta}$
- Then, the GAN training problem is

$$\min_{\theta} \text{Distance}(p_{\theta}, p^*)$$

- In the usual GAN, the distance function is the JS-divergence.

Continuity and Distance

- It is our hope to minimize $\text{Distance}(p_{\theta}, p^*)$ by gradient descent.

$$\theta \leftarrow \theta - \epsilon \nabla_{\theta} \text{Distance}(p_{\theta}, p^*)$$

- For this to work, it is essential that the following continuity property be satisfied:
- If we change θ by a little bit, then $\text{Distance}(p_{\theta}, p^*)$ changes also by only a little bit.
- It turns out, that the JS-divergence (or KL-divergence) does not satisfy this property!

Modifying the GAN Problem

- The natural question is then: can we replace JS-divergence by another measure of distance so that the continuity issue is resolved?
- The answer is yes!
- Consider the *Wasserstein distance*, or the earth-mover's distance (EMD):

$$W(p, q) = \inf_{\gamma \in \Pi(p, q)} \mathbb{E}_{(x, y) \sim \gamma} [\|x - y\|]$$

- where $\Pi(p, q)$ is the set of joint distributions whose marginals are p and q respectively.
- i.e. $\gamma \in \Pi(p, q)$ if and only if

$$\int \gamma(x, y) dy = p(x), \quad \int \gamma(x, y) dx = q(y)$$

Wasserstein GAN

- The Wasserstein GAN transforms the problem

$$\min_{\theta} D_{JS}(p_{\theta}\|p^*) \rightarrow \min_{\theta} W(p_{\theta}, p^*)$$

- Advantages:
 - Unlike D_{JS} (or D_{KL}), W is a true distance (metric).
 - The mapping $\theta \mapsto W(p_{\theta}, p^*)$ is continuous.
 - The mapping $\theta \mapsto W(p_{\theta}, p^*)$ is differentiable almost everywhere if $\theta \mapsto p_{\theta}$ is Lipschitz.

How to Compute W ?

- We need to compute

$$W(p_{\theta}, p^*) = \inf_{\gamma \in \Pi(p_{\theta}, p^*)} \mathbb{E}_{(x, y) \sim \gamma} [\|x - y\|]$$

- And possibly also $\nabla_{\theta} W(p_{\theta}, p^*)$ for GD.
- Direct computation via optimization is *not* tractable.
- However, we have a beautiful duality theory for W !

Kantorovich-Rubinstein Duality

-

$$W(p_{\theta}, p^*) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p_{\theta}} [f(x)] - \mathbb{E}_{x \sim p^*} [f(x)]$$

- Here, $\|f\|_L$ denotes the Lipschitz constant for f
- (Think maximum value of $\|\nabla f\|$)
- **Intuition:** Instead of explicitly describing how mass moves, look for a function f that separates the two distributions as much as possible, but is not allowed to change too rapidly.

Wasserstein GAN

- Owing to Kantorovich duality, we can approximate

$$W(p_{\theta}, p^*) \approx \max_{\phi} \mathbb{E}_{x \sim p_{\theta}} [f(x; \phi)] - \mathbb{E}_{x \sim p^*} [f(x; \phi)]$$

- With $f(x, \phi)$ a neural network (parameters ϕ)
- Subject to being 1-Lipschitz
- (Implemented via weight clipping or gradient penalty)
- Call f the *critic*

Gradient for Generator

- If ϕ attains the maximum, then

$$\nabla_{\theta} W = \nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}} [f(x; \phi)]$$

- Using reparameterization:

$$= \mathbb{E}_{z \sim p_0} \nabla_{\theta} f(G(z; \theta); \phi)$$

- The expectation can be computed using Monte-Carlo approximation.

Wasserstein GAN Algorithm

- Repeat:
 - For k steps (train critic):
 - Sample m noise samples $\{z^{(i)}\} \sim p_0$
 - Sample m real samples $\{x^{(i)}\} \in \mathcal{D}$
 - Update

$$\phi \leftarrow \phi + \epsilon_1 \frac{1}{m} \sum_{i=1}^m \nabla_{\phi} \left[f(x^{(i)}; \phi) - f(G_{\theta}(z^{(i)}); \phi) \right]$$

- Train generator:
 - Sample $\{z^{(i)}\} \sim p_0$
 - Update

$$\theta \leftarrow \theta - \epsilon_2 \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} f(G_{\theta}(z^{(i)}); \phi)$$

Neural Style Transfer

- The style transfer task:
 - Given a content image x_c
 - And a style image x_s
 - Generate image x with content x_c and style x_s

Notation

- Pass input image x into an L -layer CNN
- Feature maps:

$$h^{(\ell+1)} = f^{(\ell)}(h^{(\ell)}; \theta^{(\ell)}), \quad h^{(0)} = x$$

- $f^{(\ell)}$ is the ℓ -th convolutional block
- Hidden feature maps are $\{h^{(\ell)}\}$
- Entry notation:

$$h_{ij}^{(\ell)} = j\text{-th pixel of } i\text{-th channel at layer } \ell$$

- Dependence on input:

$$h_{ij}^{(\ell)} = h_{ij}^{(\ell)}(x)$$

Combining Content and Style Losses

- Goal: generate image x with content x_c and style x_s
- Optimization problem:

$$\min_x \alpha L_C(x, x_c) + \beta L_S(x, x_s)$$

- Where:
 - L_C = content loss
 - L_S = style loss

Content Loss

- Define per-layer content loss:

$$E_C(x, x_c, \ell) = \frac{1}{2} \sum_{i,j} \left(h_{ij}^{(\ell)}(x) - h_{ij}^{(\ell)}(x_c) \right)^2$$

- Total content loss:

$$L_C(x, x_c) = \sum_{\ell \in \ell_C} E_C(x, x_c, \ell)$$

- ℓ_C = set of content layers

Style Loss

- Define Gram matrix:

$$G = XX^T$$

- Measures correlations between features
- Layer-wise style loss:

$$E_S(x, x_s, \ell) = \frac{1}{4N_{\ell}M_{\ell}} \sum_{i,j} \left(G_{ij}^{(\ell)}(x) - G_{ij}^{(\ell)}(x_s) \right)^2$$

- Gram matrix:

$$G_{ij}^{(\ell)}(x) = \sum_k h_{ik}^{(\ell)}(x) h_{jk}^{(\ell)}(x)$$

- Total style loss:

$$L_S(x, x_s) = \sum_{\ell \in \ell_S} E_S(x, x_s, \ell)$$

Lecture 11

Sequence-to-sequence Modelling

- Standard RNN architecture assumes fixed input/output alignment. However, input and output sequence lengths might differ
- Given an input sequence:

$$x = (x_1, x_2, \dots, x_T)$$

Task is to predict a corresponding output sequence:

$$y = (y_1, y_2, \dots, y_{T'})$$

Applications

Examples where input and output lengths are different

- Translation between languages
 - Input = English sentence
 - Output = French sentence
- Text summarization
 - Input = Long document
 - Output = Summary of document

Encoder Decoder Architecture

Need to decouple input and output sequences. The encoder-decoder model forms a two-stage dynamical system:

- Encoder** dynamics compress the input sequence into a representation state
- Decoder** dynamics generate a new sequence conditioned on the representation state

RNN Encoder

- Encoder** processes the input

$$x = (x_1, \dots, x_T)$$

using an RNN with no outputs

$$h_t = f(x_t, h_{t-1})$$

- x_t : input token or embedding
- h_t : encoder hidden state
- <bos> or <eos>: beginning/start of sequence token
- <eos>: end of sequence token

Context Vector

- The context vector summarizes the entire input sequence

$$c = q(h_1, \dots, h_T)$$

- Simplest choice is to just use the final hidden state

$$c = h_T$$

RNN Decoder

- Decoder** RNN generates output sequentially

$$s_t = g(\hat{y}_{t-1}, s_{t-1}, c)$$

$$\hat{y}_t \sim P(\hat{y}_t \mid s_t)$$

- s_t : decoder hidden state, initial $s_0 = c$
- $\hat{y}_{t-1}$: previously generated token, initial

$$\hat{y}_0 = \text{<bos>}$$

- The decoder is a conditional language model conditioned on the encoded representation
- Decoder** RNN generates output sequentially

$$s_t = g(\hat{y}_{t-1}, s_{t-1}, c)$$

$$\hat{y}_t \sim P(\hat{y}_t \mid s_t)$$

- Output layer $\hat{y}_t$ is simply softmax regression on hidden states s_t , producing a probability vector with length equal to the target vocabulary size V

Training seq2seq Autoencoders

Probabilistic Formulation

The goal is to model the conditional probability

$$P(y = (y_1, \dots, y_{T'}) \mid x = (x_1, \dots, x_T))$$

$$= \prod_{t=1}^{T'} P(y_t \mid y_{<t} = (y_1, \dots, y_{t-1}), x)$$

The decoder approximates this distribution using its hidden state

$$P(y_t \mid y_{<t}, x) = P(y_t \mid s_t)$$

where

$$s_t = g(y_{t-1}, s_{t-1}, c)$$

and

$$c = \text{Encoder}(x)$$

Maximizing log-likelihood

Training typically minimizes

$$- \sum_{t=1}^{T'} \log P(y_t \mid y_{<t}, x)$$

which corresponds to maximizing the log-likelihood.

Training difficulty in autoregressive models

Autoregressive models can be difficult to train directly.

During generation, the decoder receives its previous prediction.

$$s_t = g(\hat{y}_{t-1}, s_{t-1}, c)$$

where

$$\hat{y}_{t-1} = \operatorname{argmax}_y P(y \mid s_{t-1})$$

Early mistakes would propagate through time.

Teacher Forcing (training)

During training, instead we feed the ground truth token delayed by one time step.

$$s_t = g(y_{t-1}, s_{t-1}, c)$$

Here, y_{t-1} are the token labels of the target sequence.

Why Teacher Forcing?

Teacher Forcing converts sequence generation into a collection of conditional next-token prediction problems.

This allows the decoder to more effectively learn the conditional distribution

$$P(y_t \mid y_{<t}, x)$$

We get more stable and faster convergence

Caveats of Teacher Forcing

- The model may become “too reliant on the teacher” and performs less well during inference
 - Phenomenon known as *exposure bias*
- A midway solution is to occasionally feed the decoder with the model’s own predicted label during training
- The proportion of predicted labels being used would gradually increase over time, so that training is still stable initially
 - The technique is called *scheduled sampling*

Beam Search

Decoding as a Search Problem

During inference, we want to generate a sequence

$$\hat{y} =_y P(y \mid x)$$

$$=_y \prod_{t=1}^{T'} P(y_t \mid y_{<t}, x)$$

If the target’s vocabulary size is V , the number of possible sequences y of length T' is $V^{T'}$

Greedy Search

So far in our model, we applied Greedy Search to choose the most probable token at each step

$$\hat{y}_t =_y P(y \mid y_{<t}, x)$$

Only need to evaluate $P(y \mid s_t)$ once per time step, so computational cost is $O(T'V)$

However, this could lead to suboptimal sequences being generated

Greedy Search Example

	1	2	3	4
<i>A</i>	0.5	0.1	0.2	0.0
<i>B</i>	0.2	0.4	0.2	0.2
<i>C</i>	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

- Greedy Search outputs $(A, B, C, \text{<eos>})$ which has likelihood $= 0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$
- A better sequence $(A, C, B, \text{<eos>})$ is possible with likelihood $= 0.5 \times 0.3 \times 0.6 \times 0.6 = 0.054$

Exhaustive Search

Bad news: The only way to guarantee finding the optimal sequence is to perform Exhaustive Search.

That is, to compare the likelihoods of all $V^{T'}$ possible sequences.

Clearly, this is too expensive for long sequences.

Beam Search (1)

Beam Search improves greedy search by keeping multiple candidate sequences

Idea:

- Maintain the top k (beam width) candidate partial sequences
 - $k = 1$ is just Greedy Search

Algorithm:

- At each time step, expand each k partial sequence with all V possible tokens to obtain kV sequences
- Prune the kV expanded sequences to keep only the top k ones

Beam Search Example

Beam width = 2, Vocabulary size $V = 5$

Candidate sequences: A, C, AB, CE, ABD, CED

Beam Search (2)

At each step, kV candidates are evaluated.

Computational cost: $O(kVT')$

- Still linear w.r.t T'
- Better search quality than Greedy Search

Scoring (to be maximized) candidate sequences of length L :

$$\frac{1}{L^\alpha} P(y_1, \dots, y_L \mid x) = \frac{1}{L^\alpha} \sum_{t=1}^L \log P(y_t \mid y_{<t}, x)$$

- Normally, $\alpha = 0.75$
- This factor would favour long sequences; otherwise, they would never be chosen when compared to their subsequences

Is One Context Vector Enough?

Last hidden State as Context Vector

- The context vector summarizes the entire input sequence

$$c = q(h_1, \dots, h_T)$$

- So far, we just used

$$c = q(h_1, \dots, h_T) = h_T$$

Last Hidden State Bottleneck

$$c = q(h_1, \dots, h_T) = h_T$$

Problem

- Bad for long input sequence since we saw from lecture 5 that RNNs are not good for modelling long range dependencies
- **Want more direct connections** from each input token to each output token

Average Pooling

$$c = q(h_1, \dots, h_T) = \frac{1}{T} \sum_{i=1}^T h_i$$

- Overcomes long-range limitations of RNN
- Decoder’s initial hidden state is now separate from encoder
 - Could be a learnable parameter

Average Pooling Limitation

Problem

- Same weight applied to every input token
- **Different output tokens should be related to different input tokens**
 - E.g. “Ich” most related to “I”. “apfel” most related to “apple”

Attention Mechanisms

Attention as Adaptive Pooling

Instead of one fixed context vector, **compute a different one for each decoding time step.**

$$c_t = \sum_i w_{t,i} h_i$$

- Decoder adapts weights $w_{t,i}$ to decide which encoder tokens to pay more attention to

Softmax on Alignment Scores

$$c_t = \sum_i w_{t,i} h_i$$

- Softmax to make $w_{t,i}$ a distribution,

$$w_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}$$

- $e_{t,i}$ are **alignment scores** measuring the relevance of encoder state $h^{(i)}$ at decoder timestep t

Alignment Scores

The alignment scores $e_{t,i}$ must be **computed from information available up to time t** . Normally implemented as

$$e_{t,i} = a(h_i, s_{t-1})$$

Typical choices (variables in red are learned)

- Dot product attention

$$e_{t,i} = s_{t-1}^T h_i$$

- General bilinear attention

$$e_{t,i} = s_{t-1}^T W h_i$$

- Additive/Bahdanau attention

$$e_{t,i} = v^T \tanh(W_s s_{t-1} + W_h h_i)$$

Query-Key-Value Attention

$$e_{t,i} = a(h_i, s_{t-1})$$

$$c_t = \sum_i w_{t,i} h_i$$

generalize

$$e_{t,i} = a(k_i, q_t)$$

$$c_t = \sum_i w_{t,i} v_i$$

Query-Key-Value Attention

- $k_i = W_k h_i$: **key**
 - Describes content available at each input position
 - E.g. for the token ‘apple’ we may want a more granular representation as ‘food item’
- $q_t = W_q s_{t-1}$: **query**
 - Represents information the decoder is seeking

Query-Key-Value Attention

- $v_i = W_v h_i$: **value**
 - Represents the information retrieved from the encoder
- Setting $W_q = W_k = W_v = I$ to be the identity matrix would just recover adaptive attention pooling that we discussed
- **Remark:** This general framework may have redundant trainable parameters (red font) with some choices of $a(\cdot, \cdot)$

“Alignments” Example: Bahdanau, Cho, Bengio

Plot of $w_{t,i}$

- Heatmap with white colour representing higher values
- White spots not necessarily on diagonal, since ordering of words can differ between languages

Multi-Head Attention

Single Query-Key-Value Attention

- Single attention mechanisms compute one weighted combination of values
- However, different aspects of the sequence may require different attention patterns

Multi-Head Attention

- **Idea:** Use multiple independent attention mechanisms, called **heads**

$$e_{t,i} = a(k_i, q_t)$$

$$w_{t,i} = \text{softmax}(e_{t,i})$$

$$c_t = \sum_i w_{t,i} v_i$$

$$e_{t,i}^l = a(k_i^l, q_t^l)$$

$$w_{t,i}^l = \text{softmax}(e_{t,i}^l)$$

$$c_t^l = \sum_i w_{t,i}^l v_i^l$$

Multi-Head Attention

- Each attention head will have an **independent set of trainable parameters** W_k^l, W_v^l, W_q^l
- They learn different projections of the input, corresponding to different aspects
- Context vectors from each head are concatenated and passed to the decoder

NLP Applications

Attention-based encoder-decoder models are very successful in many sequence conversion problems

Examples

- Machine translation
- Speech recognition
- Text Summarization

Image Captioning (Xu et al., 2016)

Input: Image

Output: Natural language description

Encoder: Convolutional neural network

Self-Attention

Recurrence of the Encoder

- Encoder **recurrently** produces input hidden states
- Hidden state for an input token is influenced by all preceding input tokens
 - i.e. decoder pays attention to the subsequence up to this time, not just the word

Is Recurrence Necessary?

- With attention mechanism, the decoder should figure out which specific input token/word to attend to at each time
- **Idea: Remove the recurrence**

Non-recurrent Encoder

Problem: Encoder loses **context-specificity**

- e.g. the embedding for “an” should really depend on the other words in the input sequence
 - It could be translated to “ein”, “einer” or “eines” depending on the context

Non-recurrent Encoder

Problem: Encoder loses **context-specificity**

Solution: Apply attention within encoder itself, to allow input words to pay attention to each other

Self-Attention (Encoder)

$$q_i = W_q h_i$$

$$k_i = W_k h_i$$

$$v_i = W_v h_i$$

- **Hidden states:** For now, just word embeddings

- Query, Key and Value are all projections of input tokens with independent learnable weight matrices W_q, W_k, W_v

Remark: Queries and keys are interchangeable here

Self-Attention

$$q_i = W_q h_i$$

$$k_i = W_k h_i$$

$$v_i = W_v h_i$$

$$w_{ij} = \text{attn}(q_i, k_{0:N})$$

$$e_{ij} = q_i^T k_j$$

$$w_{i0}, \dots, w_{iN} = \text{softmax}(e_{i0}, \dots, e_{iN})$$

- **Dot-product attention** $e_{ij} = q_i^T k_j$ is chosen

- A softmax operation is applied to obtain the attention weights of input words w.r.t each other

Self-Attention

$$q_i = W_q h_i$$

$$k_i = W_k h_i$$

$$v_i = W_v h_i$$

$$w_{ij} = \text{attn}(q_i, k_{0:N})$$

$$o_i = \sum_j w_{ij} v_j$$

Updated representation o_i for the word i is the **attention-weighted sum of the values for all words, including itself**
Self-Attention

$$w_{0j} = \text{attn}(q_0, k_{0:N})$$

$$o_0 = \sum_j w_{0j} v_j$$

Updated representation o_i for the word i is the **attention-weighted sum of the values for all words, including itself**
Single-head Self-Attention

$$q_i = W_q h_i$$

$$k_i = W_k h_i$$

$$v_i = W_v h_i$$

$$w_{ij} = \text{attn}(q_i, k_{0:N})$$

$$o_i = \sum_j w_{ij} v_j$$

This is what we call a “single-head” Self-Attention Block

Multi-Head Self-Attention

$$q_i^a = W_q^a h_i$$

$$k_i^a = W_k^a h_i$$

$$v_i^a = W_v^a h_i$$

$$w_{ij}^a = \text{attn}(q_i^a, k_{0:N}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$

$$o_i = [o_i^1; o_i^2; o_i^3; \dots; o_i^H]$$

Multiple parallel self-attention “heads”, each with

- **Independent queries, keys and values** with its own learnable parameters W_q^a, W_k^a, W_v^a
- This results in independent attention weights focusing on different aspects on the input sequence

Multi-Head Self-Attention

$$q_i^a = W_q^a h_i$$

$$k_i^a = W_k^a h_i$$

$$v_i^a = W_v^a h_i$$

$$w_{ij}^a = \text{attn}(q_i^a, k_{0:N}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$

$$o_i = [o_i^1; o_i^2; o_i^3; \dots; o_i^H]$$

Multiple self-attention “heads”, each with

- **Independent outputs** o_i^a
- Final output o_i is the concatenation of the outputs from each head

Multi-Head Self-Attention + MLP

$$q_i^a = W_q^a h_i$$

$$k_i^a = W_k^a h_i$$

$$v_i^a = W_v^a h_i$$

$$w_{ij}^a = \text{attn}(q_i^a, k_{0:N}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$

$$o_i = [o_i^1; o_i^2; o_i^3; \dots; o_i^H]$$

$$y_i = \text{MLP}(o_i)$$

- Normally, the output of Multi-head Self-attention is passed through some FCNN/MLP layers
- **Note:** the same MLP is applied to all output tokens (i.e. shared parameters)
- Entire unit is called a **Multi-head Self-attention Block**

Stacking blocks

We often stack several Multi-head Self Attention Blocks

Positional Encoding

Self-Attention Block Symmetries without Positional Encoding

The Self-Attention Blocks defined so far is a **permutation equivariant function!** i.e. If we permute the input tokens, the output would be permuted the same way. The output representation o_i of a particular word is invariant to the ordering of other words in the sequence.

- In particular, **relative position of words is ignored**
- Adjacent word is treated the same as a word in the next paragraph

Self-Attention Block with Positional Encoding

Goal: Design a method to **encode positional information** that captures the **relative position of tokens**

Hidden State h_i = Word Embedding + Positional Encoding

Relative Position

Desirable properties

- **Only relative distance matters**
 - The attention which vector v_{t+T} pays to vector v_t is independent of T
- **Attention falls off with distance**, so that adjacent vectors influence each other more than faraway vectors

Positional Encoding

$$P_t = \begin{bmatrix} \sin \omega_1 t \\ \cos \omega_1 t \\ \sin \omega_2 t \\ \cos \omega_2 t \\ \vdots \\ \vdots \\ \sin \omega_{d/2} t \\ \sin \omega_{d/2} t \end{bmatrix}$$

$$\omega_l = \frac{1}{10000^{2l/d}}$$

$$P_{t+\tau} = M_\tau P_t$$

$$M_\tau = \text{diag} \left(\begin{bmatrix} \cos \omega_l \tau & \sin \omega_l \tau \\ -\sin \omega_l \tau & \cos \omega_l \tau \end{bmatrix}, l = 1 \dots d/2 \right)$$

Linear relationship due to double angle formula:

- **Exercise:** Check that $P_{t+\tau} = M_\tau P_t$
- **Recall:** $\sin(A+B) = \sin A \cos B + \cos A \sin B$

Positional Encoding Properties -Relative Position

$$P_t = \begin{bmatrix} \sin \omega_1 t \\ \cos \omega_1 t \\ \sin \omega_2 t \\ \cos \omega_2 t \\ \vdots \\ \vdots \\ \sin \omega_{d/2} t \\ \sin \omega_{d/2} t \end{bmatrix}$$

$$\omega_l = \frac{1}{10000^{2l/d}}$$

$$P_{t+\tau} = M_\tau P_t$$

$$M_\tau = \text{diag} \left(\begin{bmatrix} \cos \omega_l \tau & \sin \omega_l \tau \\ -\sin \omega_l \tau & \cos \omega_l \tau \end{bmatrix}, l = 1 \dots d/2 \right)$$

- Every vector P_t is **unique** and equal norm
 - Relationship between P_t and $P_{t+\tau}$ only depends on their **relative distance** $P_{t+\tau} = M_\tau P_t$
 - **Linear relationship** enables the model to learn **shift-invariant “gap” dependent relationships**
- Positional Encoding Properties -Absolute Position**
- In binary, a **higher bit has lower frequency** than a lower bit
 - Heatmap of positional encoding shows the same property
 - **Binary vectors have non-linear relationships**, and representations are **discontinuous** w.r.t distance. E.g.
 - 0111111111
 - 1000000000

Masked Self-Attention

Self-Attention for Decoder?

- **Self-attending encoder removes recurrence** while maintaining context specificity
- **Can we do the same for the decoder?**
 - i.e. does self-attention make sense in the decoder?

Self-Attention in Encoder

$$q_i = W_q h_i$$

$$k_i = W_k h_i$$

$$v_i = W_v h_i$$

$$w_{ij} = \text{attn}(q_i, k_{0:N})$$

$$o_i = \sum_j w_{ij} v_j$$

- **Self-Attention** allows the hidden state to be influenced by hidden states at all time steps, **including future time**.
- This is okay in the encoder, as inputs for all time are available

Masked Self-Attention

Decoder is sequential

- Each word is produced using the previous predicted word as input

Decoder is sequential

- Only hidden states at time t are available to compute output at time t

Self-Attention for decoder

- The attention has to be **“masked”**, forcing weights at time $t+1$ and later to be zero

Masked Self-Attention Layer

$$q_i = W_q h_i$$

$$k_i = W_k h_i$$

$$v_i = W_v h_i$$

$$e_{ij} = q_i^T k_j$$

$$w_{i0}, \dots, w_{ii} = \text{softmax}(e_{i0}, \dots, e_{ii})$$

$$o_i = \sum_{j=0}^{i-1} w_{ij} v_j$$

The **Masked Self-Attention layer** sequentially computes outputs from left to right, always **masking future outputs** when computing attention weights

Masked Self-Attention Block

$$q_i = W_q h_i$$

$$k_i = W_k h_i$$

$$v_i = W_v h_i$$

$$e_{ij} = q_i^T k_j$$

$$w_{i0}, \dots, w_{ii} = \text{softmax}(e_{i0}, \dots, e_{ii})$$

$$o_i = \sum_{j=0}^{i-1} w_{ij} v_j$$

The **Masked Self-Attention block** stacks an MLP after the masked self-attention layer

- Just as we did with the encoder self-attention block

Masked Self-Attention Block

The **Masked Self-Attention block** sequentially computes outputs from left to right

Sequential nature of decoding prevents parallel computing of outputs during inference

- Still parallelizable if we use teacher forcing during training

Masked Multi-head Self-attention

Masked Multi-head Self-attention Block

$$q_i^a = W_q^a h_i$$

$$k_i^a = W_k^a h_i$$

$$v_i^a = W_v^a h_i$$

$$w_{ij}^a = \text{attn}(q_i^a, k_{0:i-1}^a)$$

$$o_i^a = \sum_j w_{ij}^a v_j^a$$

- Multiple masked self-attention heads, **concatenating outputs from each head**
- An MLP layer follows

Encoder-Decoder Transformer Architecture (Encoder)

Encoder: Stack several **Multi-head Self-attention blocks** to get a good embedding of **input sequence**

Encoder-Decoder Transformer Architecture (Decoder)

Decoder: Stack several **Masked Multi-head Self-attention blocks** to get a good embedding of **output sequence**

Encoder-Decoder Transformer Architecture (Encoder to Decoder)

Cross-attention: Multi-head attention with

- **Queries** = Linear Projection [Decoder states]
 - **Keys and Values** = Linear Projection [Encoder outputs]
- “Attention is all you need” Vaswani, Ashish, et al.**
- “Feed Forward” was what we called MLP layers
 - **Reshape** input shape

$$(batch\ size, seq\ length, feature\ size)$$

into

$$(batch\ size * seq\ length, feature\ size)$$

- **Apply two layer MLP**
- **Reshape** back to 3D
- **Equivalent to applying two (1,1) convolution layers**

“Attention is all you need” Vaswani, Ashish, et al.

- “Add & Norm” layers are just **residual layers** and **layer-norm**
- **Input shape** (*batch size, seq length, feature size*)
- **Layer-norm** computes mean and variances across **last dimension**
 - **Trainable parameters of length = feature size**
- Batch-norm computes across first dimension
- **Every token has same mean and variance after layer-norm**, stabilizing dot-product attention scales

“Attention is all you need” Vaswani, Ashish, et al.

- **Transformers have better performance, with lower computational cost for large problems**
- For small problems/hardware, LSTMs are still better
- E.g. Phones, IoT etc

Self Attention Matrix Computations
Self-attention can be written compactly using matrices

$$X = \begin{bmatrix} x_1 \\ \vdots \\ x_T \end{bmatrix}$$

- $Q = XW_q$
- $K = XW_k$
- $V = XW_v$

Attention scores

- $E = QK^T$

Attention weights (row-wise softmax)

- $A = \text{softmax}(E)$

Output representations

- $Z = AV$

Parallel Computing for Self-Attention

Self-Attention removes the recurrence in RNNs

- This allows for **parallel computation**, as hidden states do not need to be computed in sequence
 - No need to wait for previous hidden state to be computed before computing next one
- $E = QK^T$ **computes interactions between all tokens simultaneously**
- **Computational cost** = $O(N^2)$
 - N = sequence length
 - But this is parallelizable

Parallel Computing for Masked Self-Attention

Masked Self-Attention is only parallelizable during training if we use teacher-forcing.

The trick is to apply a **causal mask** M where

$$M_{ij} = -\infty \text{ if } j > i, 0 \text{ otherwise}$$

Masked self-attention is computed using

$$\text{MaskedAttn}(Q, K, V) = \text{softmax}(QK^T + M)V$$

- **Not parallelizable during inference**, because need to wait for previous tokens to be output
- **Same story for cross-attention**

Scaled Dot-product Attention

- In **dot-product attention**, alignment scores are computed using the dot product

$$e_{i,j} = q_i^T k_j$$

- If the vector dimension $\dim(q_i) = d_k$ is **large, these values can grow large**
 - If q and k have zero-mean, variance = 1
 - $\text{Var}(q^T k) = d_k$
- **This causes softmax to have sharp peaks**

$$w_{i,j} = \text{softmax}(e_{i,j})$$

- Small gradients, slow training

Scaled Dot-product Attention

Solution:

$$e_{i,j} = \frac{q_i^T k_j}{\sqrt{d_k}}$$

- **Constant variance** on average even as d_k increases

Matrix Equations for Scaled Dot-Product Attention

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

Remark: Layer-norm also helps with this

Vision Transformer

Transformers for images?

Idea: Treat an image as a **sequence of patches**, analogous to tokens in NLP

- **Global receptive field** achieved with first layer already
- **Long-range dependencies modelled by attention**

Patch Embedding

Input image

$$x \in \mathbb{R}^{H \times W \times C}$$

Patch size

$$P \times P$$

Number of patches

$$N = \frac{HW}{P^2}$$

Each patch corresponds to a token in a sequence

$$x_i \in \mathbb{R}^{P \times P \times C}$$

Patch Embedding

Each **flattened patch** $x_i \in \mathbb{R}^{P^2 C}$ is **projected to a vector** via a learnable linear projection

$$z_i = Ex_i \in \mathbb{R}^D$$

Where

$$E \in \mathbb{R}^{D \times P^2 C}$$

Stack all tokens

$$z = (z_1, z_2, \dots, z_N) \in \mathbb{R}^{N \times D}$$

Image x transformed into a length N sequence z

Equivalent: Convolution with filter size $P \times P$, stride length P , D filters/output channels

Patch Embedding Example

In the original ViT vision transformer paper,

Image size: 224×224

Patch size: 16×16

Number of patches:

$$N = \frac{224 \times 224}{16 \times 16} = 196$$

Class Token

- **Learnable vector**

$$z_{\text{cls}} \in \mathbb{R}^d$$

prepended to sequence

$$z \in \mathbb{R}^{N \times D}$$

- **CLS token aggregates global information**

Positional Embedding (ViT)

- **Learnable parameters**

$$E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$$

- Encode **spatial locations** of patches
- Usually fixed resolution (no extrapolation needed)

Final input sequence

$$[z_{\text{cls}}, z] + E_{\text{pos}}$$

Transformer Encoder (ViT)

- After patch embedding + positional encoding:
- Pass through a **transformer encoder stack**
- Uses **pre-normalization** (LN before sublayers)

Pre-norm vs Post-norm

$$\text{Post-norm: } x_{t+1} = \text{Norm}(x_t + F_t(x_t))$$

$$\text{Pre-norm: } x_{t+1} = x_t + F_t(\text{Norm}(x_t))$$

- Post-norm: tighter control, harder to train
- Pre-norm: **more stable gradients**, easier training
- Modern deep transformers use **pre-norm**

Output Representation (ViT)

- CLS representation after n layers:

$$\text{Rep}_{\text{CLS}}$$

- Used as input to **classification MLP**
- Produces final **probability output**

Important note

- Without positional encoding $\rightarrow$ model is **permutation invariant**

ViT Scaling Observation

- ViT performs worse than CNNs on **small datasets**
- With large-scale pretraining $\rightarrow$ **outperforms CNNs**

CNN vs Vision Transformer

- **CNN**
 - Strong inductive bias
 - Local convolutions
 - Needs less data
 - Receptive field grows with depth
- **ViT**
 - Weak inductive bias
 - Global attention from start
 - Needs large datasets

Key Insight

- **CNNs:**
 - Translation equivariance
 - Learn local structure efficiently
- **ViTs:**
 - Must **learn locality from data**
 - Ignore spatial structure without positional encoding

Conformer

- Combines **convolution + self-attention**
- Convolution $\rightarrow$ captures **local (short-range)**
- Attention $\rightarrow$ captures **global (long-range)**

Tokenizers

- Transformers = **token sequence processors**
- Any modality works if tokenized properly

Examples

- Text: word embeddings
- Vision: patch embeddings
- Audio: spectrogram patches
- Multimodal: concatenate tokens